

BLACK-BOX PENETRATION TEST REPORT

OWASP Juice Shop

Web Application Security Assessment

Target: juice-shop

Assessment Type: Black-Box Web Application Penetration Test

Environment: Local Docker Deployment

Tester: Sirajudin Seid

Assessment Period: September 14–20, 2026

Report Date: September 2026

1. Report Overview

This report presents the results of a black-box web application penetration test performed against a local Docker deployment of the OWASP Juice Shop. The assessment record supplied for this rewrite documents authentication, authorization, injection, sensitive-data exposure, input-validation, business-logic, cryptographic, and configuration weaknesses.

The technical findings and evidence in this document are based on the supplied penetration-test report and its embedded screenshots. The findings have been rewritten for clarity and professional presentation; they have not been independently re-tested during preparation of this document.

Numbering note

The source report contains inconsistent finding numbering, including a duplicated finding number and a missing number in the later sequence. To avoid ambiguity, this revised report assigns a unique sequential identifier (F-01 through F-33) to every distinct finding documented in the source material.

2. Assessment Scope and Approach

Scope

Target	juice-shop
Base URL used in evidence	http://localhost:3040/
Environment	Local Docker deployment
Assessment type	Black-box web application penetration test
Assessment period	September 14–20, 2026

Testing approach

- Application enumeration and endpoint discovery.
- Manual testing of authentication and authorization controls.
- HTTP request interception and modification using Burp Suite.
- Input-manipulation testing for SQL injection, XSS, and validation weaknesses.
- Inspection of client-side JavaScript and application source for exposed secrets and credentials.
- Testing of JWT construction and validation behavior.

- Testing of public files, directories, metrics, logs, image metadata, and backup artifacts.
- Validation of business-logic controls such as payment and membership enforcement.

Tools and techniques referenced in the source assessment

- Burp Suite / Burp Intruder and Repeater
- Gobuster
- Browser Developer Tools
- JWT decoding tools
- Hashcat / offline hash analysis
- exiftool
- Publicly available OSINT sources

3. Executive Summary

The assessment identified a broad set of security weaknesses affecting the application's input handling, authentication, authorization, data protection, client-side security, account recovery, and server configuration. The most consequential documented issues include SQL injection authentication bypasses, insecure JWT handling, exposed password hashes, client-side credential exposure, an exposed wallet seed phrase, unauthorized role assignment, and insufficient authorization checks.

The supplied report explicitly classifies three findings as Critical, eight as High, two as Medium, two as Low, and additional findings as mixed-severity or not explicitly rated. These classifications are reproduced from the source where provided and are not newly calculated in this rewrite.

Key security themes

- Injection vulnerabilities permit manipulation of SQL queries and browser-executed content.
- Authentication controls can be bypassed through SQL injection, weak password practices, or improperly validated JWTs.
- Authorization controls do not consistently bind requested resources and privileges to the authenticated user.

- Sensitive information, credentials, password hashes, wallet secrets, and operational logs are exposed through client-side code or public endpoints.
- Several security controls are implemented only in the client, allowing them to be bypassed by modifying requests or browser state.
- Password storage and account-recovery mechanisms do not provide adequate protection against offline cracking or targeted social engineering.

4. Findings Overview

ID	Finding	Severity	Affected Component / Endpoint
F-01	HTML Injection and DOM-Based Cross-Site Scripting (XSS)	High (XSS) / Medium (HTML Injection)	Search Bar / Search Results Display <code>http://localhost:3040/#/search?q=<payload></code>
F-02	SQL Injection in Login — Authentication Bypass	Critical	Login Functionality — Email and Password Fields POST <code>/rest/user/login</code> (UI: <code>http://localhost:3040/#/login</code>)
F-03	Sensitive Data Exposure in JWT and Weak Password Hashing	Critical	JWT Generation / Authorization Header GET <code>/rest/admin/application-configuration</code> and authenticated API endpoints
F-04	Exposed Backup Files and Malicious File Hosting	High	Public FTP Directory / File Server <code>/ftp/</code> and <code>/ftp/quarantine/</code>
F-05	Improper Error Handling and Verbose Error Disclosure	Not specified in the source report	Login / Authentication POST <code>/rest/user/login</code> (UI: <code>#!/login</code>)
F-06	SQL Injection in Product Search — Potential Full Database Compromise	Not specified in the source report	Product Search GET <code>/rest/products/search?q=</code>
F-07	Unauthorized Role Assignment During Registration	Not specified in the source report	User Registration / Account Management POST <code>/api/Users/</code> (UI: <code>#!/register</code>)
F-08	Weak Administrative Password and Insufficient Brute-Force Protection	Not specified in the source report	Login / Authentication POST <code>/rest/user/login</code> (UI: <code>#!/login</code>)
F-09	Insufficient Authorization on Basket Access	Not specified in the source report	Basket / Authorization GET <code>/rest/basket</code>
F-10	Privilege Escalation Through Improper JWT Signature Validation	Not specified in the source report	Authentication / JWT Authorization Administration-related protected endpoints (UI: <code>#!/administration</code>)
F-11	Improper Authorization in Review Author Attribution	Not specified in the source report	Product Reviews / Author Attribution POST <code>/rest/Product/{productId}/reviews</code> (UI: <code>#!/search</code>)

BLACK-BOX PENETRATION TEST REPORT — OWASP JUICE SHOP

F-12	Reflected XSS in Product Search	Not specified in the source report	Product Search GET /rest/products/search?q= (UI: /#/search)
F-13	Unauthenticated Exposure of Application Metrics	Low	Prometheus Metrics / Application Telemetry GET /metrics
F-14	Hardcoded Credentials Exposed in Client-Side JavaScript	Not specified in the source report	Client-side JavaScript / Authentication Logic Client-side JavaScript bundle (e.g., app.js/main.js)
F-15	Credential Exposure Through Publicly Available OSINT and Weak Password Construction	Not specified in the source report	User Authentication POST /rest/user/login
F-16	Forgotten Developer Backup File Exposed	Not specified in the source report	Backup Files / Web Server GET /ftp/package.json.bak (or similar backup paths)
F-17	Leaked API Key in Client-Side Code	Not specified in the source report	Client-side JavaScript / Public Configuration Client-side JavaScript files such as main.js
F-18	Improper Input Validation — Empty User Registration	Low/Medium	User Registration POST /api/Users/
F-19	Client-Side Exposure of User Credentials	Not specified in the source report	Client-side JavaScript http://localhost:3040/hacking-instructor-BXwB7EFQ.js
F-20	NFT Takeover Through Exposed Wallet Seed Phrase	Critical	About Page / Wallet Secret Handling /about
F-21	Sensitive Location Exposure Through Image EXIF Metadata	High	Photo Wall / Account Recovery /#/photo-wall and /#/forgot-password
F-22	Repetitive Registration — Insufficient Server-Side Password Confirmation	Medium	User Registration /#/register
F-23	Zero-Star Feedback — Improper Input Validation	Low	Customer Feedback POST /api/Feedbacks/ (UI: /#/contact)
F-24	Password Hash Exposure Through User Profile API	High	User Profile / API Response GET /rest/user/whoami
F-25	Visual Geo-Stalking — Sensitive Data Exposure	High	Photo Wall / Password Recovery /#/photo-wall and /#/forgot-password
F-26	Weak Password Cryptography — Use of Unsalted MD5	High	Password Storage / User Data /rest/user/whoami?fields=email,password
F-27	Login Authentication Bypass for Jim via SQL Injection	High	Login / Authentication /#/login
F-28	Login Authentication Bypass for Bender via SQL Injection	High	Login / Authentication /#/login
F-29	CAPTCHA Bypass	Medium	CAPTCHA / Feedback Submission GET /rest/captcha/ and feedback submission
F-30	Missing URL Encoding Allows Access to an Unintended Resource	Not specified in the source report	Photo Wall / Image Resource Handling Image URL/file parameter
F-31	Business Logic Bypass — Deluxe Membership Payment Validation	High	Deluxe Membership / Payment Processing POST /rest/deluxe-membership
F-32	Publicly Accessible Support Logs and Observability Data	Not specified in the source report	Support Logs / Web Server Configuration GET /support/logs

BLACK-BOX PENETRATION TEST REPORT — OWASP JUICE SHOP

F-33	Unsigned JWT Accepted by Authentication Logic	Not specified in the source report	JWT Validation Logic POST /rest/user/login and protected API requests
------	---	------------------------------------	---

5. Detailed Findings

Each finding below contains a clear description, affected component and endpoint, reproduction procedure, observed impact, remediation guidance, and the original screenshot evidence associated with that finding.

F-01: HTML Injection and DOM-Based Cross-Site Scripting (XSS)

Severity	High (XSS) / Medium (HTML Injection)
CVSS	Not specified separately in the source report
Affected Component	Search Bar / Search Results Display
Affected Endpoint	http://localhost:3040/#/search?q=<payload>

Description

The search functionality accepts user-controlled input and renders it into the page without sufficient contextual encoding or sanitization. During testing, HTML markup supplied through the search field was interpreted by the browser as HTML rather than displayed as plain text. A second test demonstrated that a JavaScript URI embedded in an injected anchor element could execute JavaScript in the victim browser.

Reproduction / Verification Steps

1. Open the OWASP Juice Shop application and locate the search bar.
2. Enter the following HTML payload: `<h1>HTML injection</h1>`.
3. Submit the search. The application renders the supplied text as an H1 element in the search-results area rather than treating it as ordinary text.
4. Repeat the test with the following payload: `<a href="javascript:alert(1)">Click Me</a>`.
5. Submit the search. A clickable link named “Click Me” is rendered in the results.
6. Click the injected link. A JavaScript alert displaying “1” is generated by the application.

Evidence

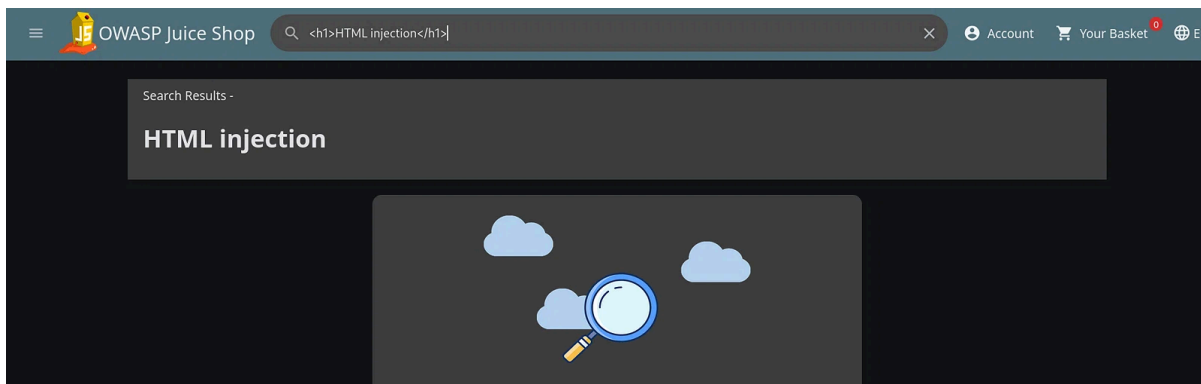


Figure 1. Evidence screenshot

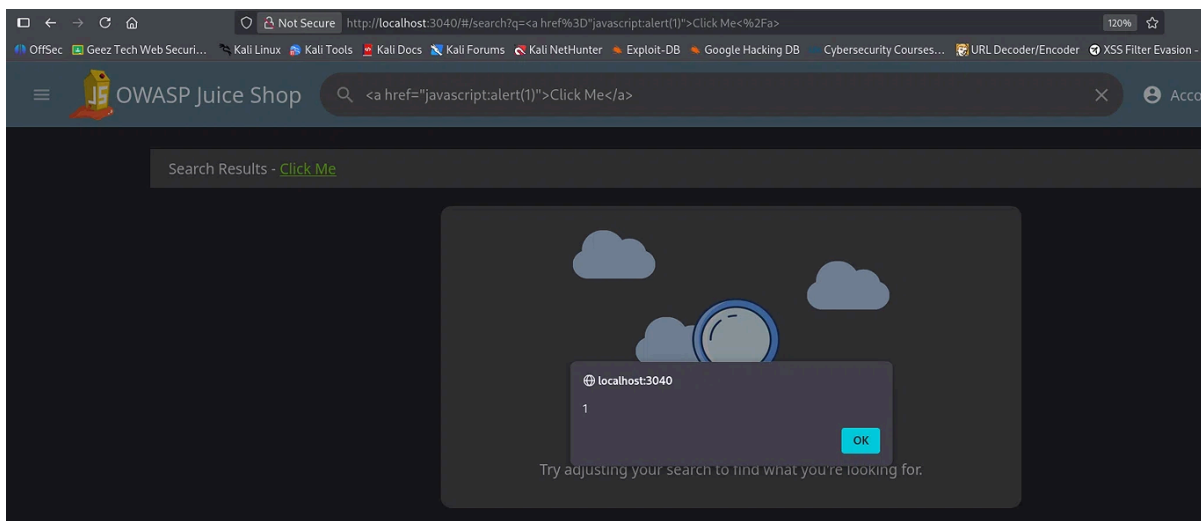


Figure 2. Evidence screenshot

Impact

- Page defacement: an attacker can alter the visual content presented to users.
- Phishing: injected HTML can be used to display fraudulent forms or malicious links.
- JavaScript execution: an attacker may execute code in the victim's browser in the application's security context.
- Depending on browser/session protections, exploitation could support session compromise, unauthorized actions, keylogging, or malicious redirection.

Recommended Remediation

1. Apply context-aware output encoding before placing user-controlled data into HTML, attributes, JavaScript, or URL contexts.

2. Use safe DOM APIs such as `textContent` instead of `innerHTML` when rendering untrusted text.
3. If HTML rendering is genuinely required, sanitize the content with a maintained library such as `DOMPurify`.
4. Implement a restrictive Content Security Policy (CSP) that limits script execution and blocks unsafe inline JavaScript.
5. Validate and sanitize the `q` parameter server-side and reject dangerous protocols or markup where they are not required.

F-02: SQL Injection in Login — Authentication Bypass

Severity	Critical
CVSS	9.8 (as stated in the source report)
Affected Component	Login Functionality — Email and Password Fields
Affected Endpoint	POST <code>/rest/user/login</code> (UI: <code>http://localhost:3040/#/login</code>)

Description

The login endpoint incorporates attacker-controlled input into a database query without adequate parameterization. A crafted value in the email field bypassed the normal password verification process and resulted in administrative authentication without valid credentials.

Reproduction / Verification Steps

1. Open the login page at `http://localhost:3040/#/login`.
2. Enter the following value in the Email field: `' OR 1=1--`.
3. Enter any arbitrary value in the Password field, such as `password` or `123456`.
4. Submit the login form.
5. The application accepts the request and redirects the tester to the administration dashboard without valid credentials.

Evidence

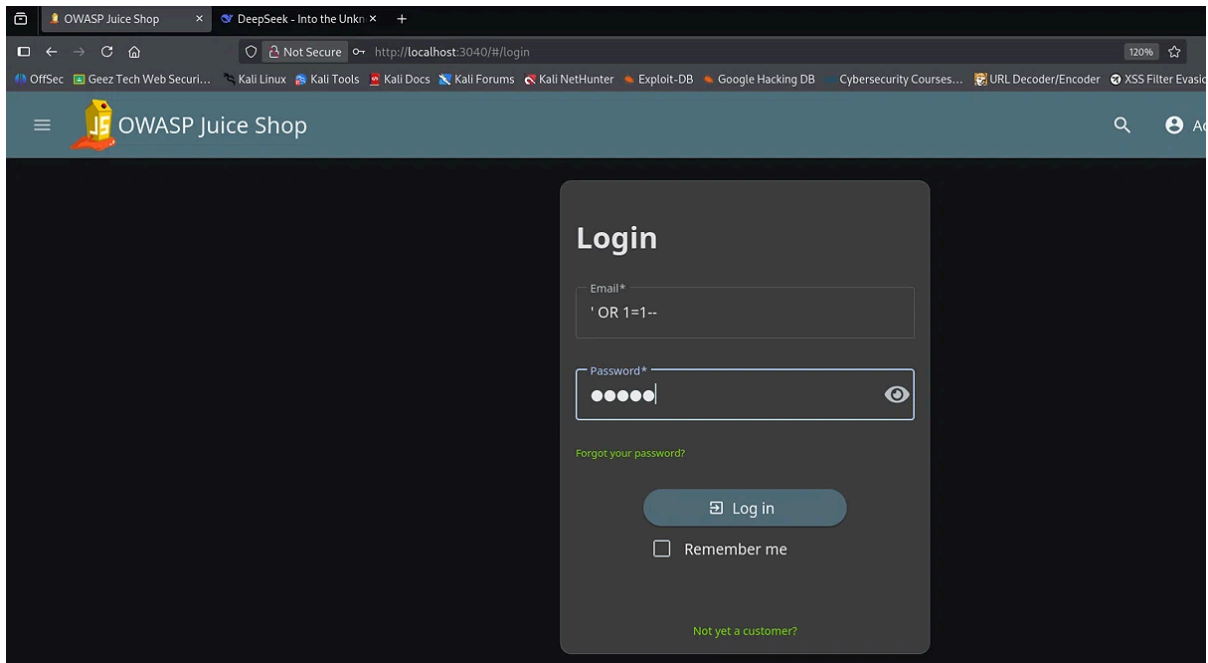


Figure 3. Evidence screenshot

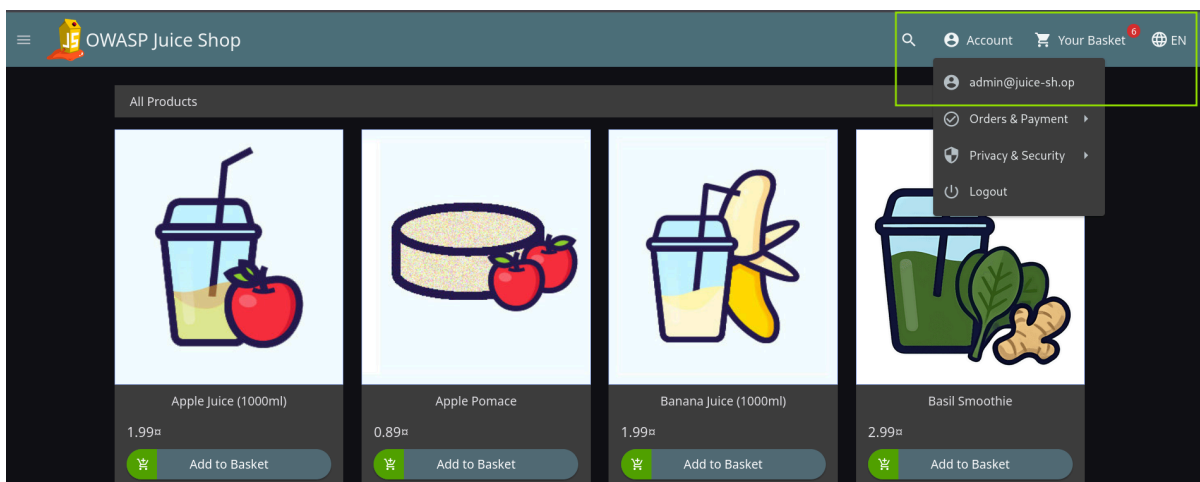


Figure 4. Evidence screenshot

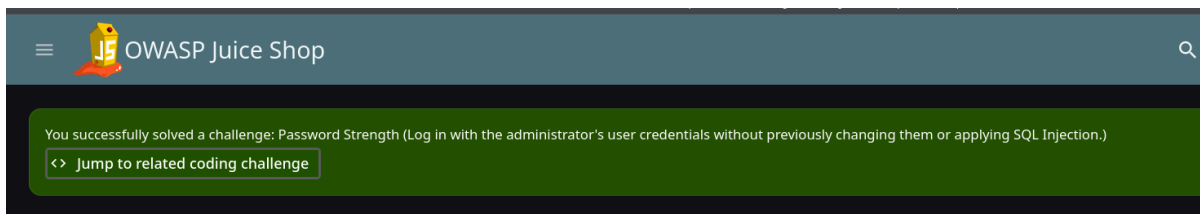


Figure 5. Evidence screenshot

Impact

- Authentication bypass: an unauthenticated attacker can access the application without knowing a valid password.
- Privilege escalation: the demonstrated bypass provides administrative access.
- Unauthorized data access: administrative access may expose user accounts, order history, payment-related information, and other sensitive application data.
- Database compromise: more advanced SQL injection techniques may allow broader data extraction or manipulation.

Recommended Remediation

1. Use parameterized queries or prepared statements for every database operation involving user input.
2. Do not construct SQL statements through string concatenation.
3. Validate input using an allow-list as a secondary control; do not treat filtering as the primary SQLi defense.
4. Apply least-privilege permissions to the application database account.
5. Use a well-maintained ORM or database library with parameterized query mechanisms enabled.
6. Consider a WAF as a defense-in-depth control, not as a replacement for secure query construction.

F-03: Sensitive Data Exposure in JWT and Weak Password Hashing

Severity	Critical
CVSS	9.1 (as stated in the source report)
Affected Component	JWT Generation / Authorization Header
Affected Endpoint	GET /rest/admin/application-configuration and authenticated API endpoints

Description

The application issues a JSON Web Token containing more information than is required for authorization. The tested token payload exposed the user's email, role, internal identifier, and an MD5 password hash. Because MD5 is a fast and obsolete password-hashing algorithm, disclosure of the hash creates a practical offline password-cracking risk.

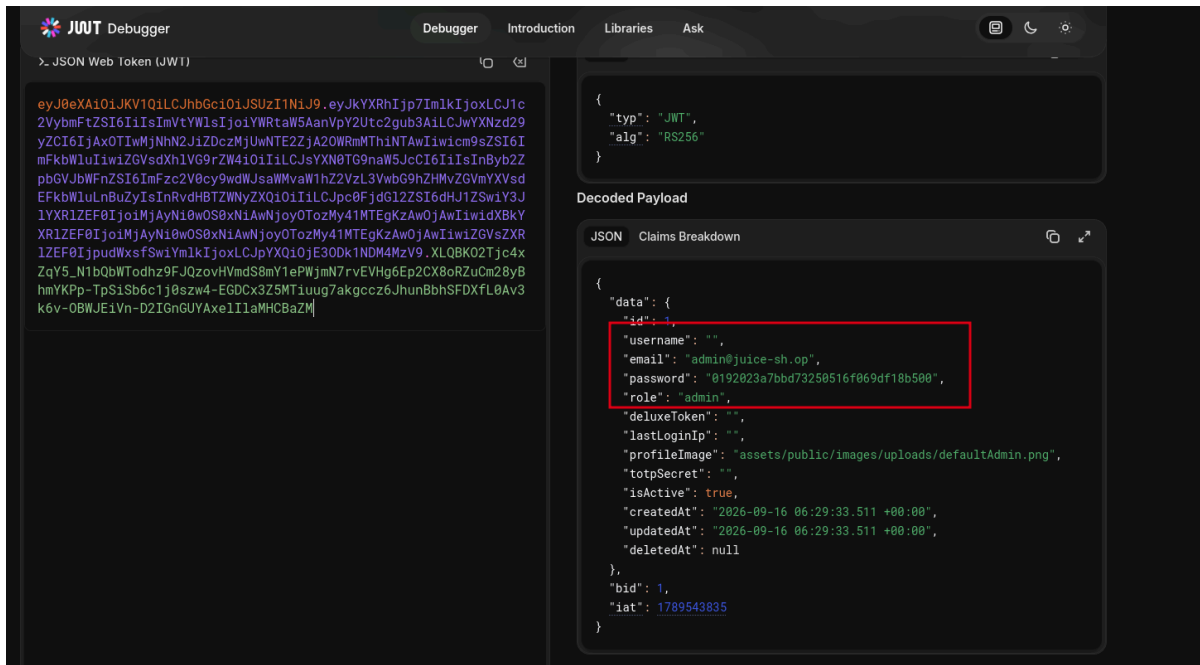


Figure 7. Evidence screenshot

Impact

- Account takeover: an attacker who obtains and cracks the exposed password hash can authenticate normally even if the SQL injection is later fixed.
- Credential reuse risk: if the same password is reused elsewhere, compromise may extend beyond the application.
- Persistent access: valid credentials can provide continued access until the password is changed.
- Privacy exposure: unnecessary disclosure of user identifiers and email addresses increases the application's data-exposure footprint.

Recommended Remediation

1. Remove passwords, password hashes, secrets, and other unnecessary sensitive data from JWT payloads.
2. Keep JWT claims to the minimum required for authorization, for example a user identifier and role where appropriate.
3. Migrate password storage from MD5 to a dedicated password-hashing algorithm such as Argon2id, bcrypt, or scrypt, using unique salts.

4. Use short-lived tokens and an appropriate revocation strategy for compromised sessions.
5. Use encrypted tokens only where there is a genuine requirement for confidential token contents; do not use token encryption as a substitute for minimizing the payload.

F-04: Exposed Backup Files and Malicious File Hosting

Severity	High
CVSS	7.5 (as stated in the source report)
Affected Component	Public FTP Directory / File Server
Affected Endpoint	/ftp/ and /ftp/quarantine/

Description

A publicly accessible FTP-style directory was discovered through robots.txt. Directory listing exposed backup files, a password-database file, and files capable of being used in social-engineering or malware-delivery scenarios. The issue combines information disclosure with an unsafe file-hosting configuration.

Reproduction / Verification Steps

1. Open the application root at <http://localhost:3040/>.
2. Request <http://localhost:3040/robots.txt>.
3. Observe that the robots.txt content reveals the /ftp directory.
4. Navigate to <http://localhost:3040/ftp/>.
5. Observe that directory listing is enabled and multiple files are accessible.

Evidence

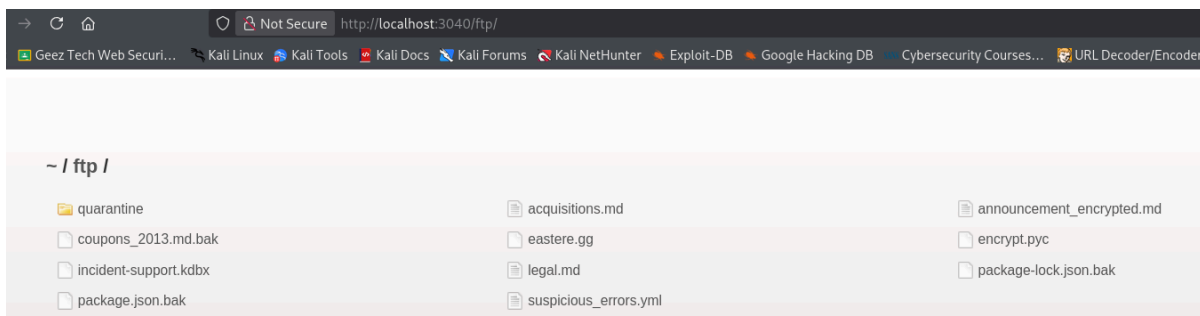


Figure 8. Evidence screenshot supplied with the original assessment record.

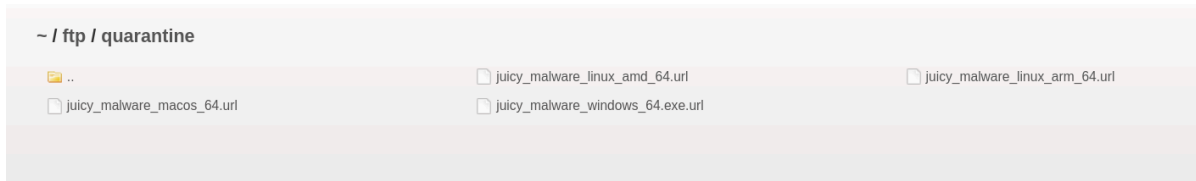


Figure 9. Evidence screenshot supplied with the original assessment record.

Impact

- Credential exposure: the exposed .kdbx file may provide material for offline credential cracking.
- Technology fingerprinting: backup package manifests can reveal dependency versions and help attackers identify known weaknesses.
- Client-side compromise risk: files hosted in the directory may be used in phishing or malicious-download scenarios.
- Application reconnaissance: exposed files reveal information about the application's structure and deployment.

Recommended Remediation

1. Remove backup files, password databases, logs, and other sensitive artifacts from web-accessible directories.
2. Disable directory listing at the web-server level.
3. Protect sensitive file-storage paths with authentication and authorization rather than relying on robots.txt.
4. Allow only approved upload types and scan uploaded content before storage or delivery.
5. Serve untrusted downloads with appropriate Content-Disposition and generic content types where applicable.

F-05: Improper Error Handling and Verbose Error Disclosure

Severity	Not specified in the source report
CVSS	Not specified
Affected Component	Login / Authentication
Affected Endpoint	POST /rest/user/login (UI: /#/login)

Description

The login endpoint was observed to return overly detailed error information when malformed input was submitted. The original assessment indicates that raw database and implementation details were exposed to the client instead of a controlled, user-safe error response.

Reproduction / Verification Steps

1. Open the login page.
2. Enter a single quote (') in the email field.
3. Enter any value in the password field.
4. Intercept the request with Burp Suite and inspect the response.
5. Observe the detailed error information returned by the application.

Evidence

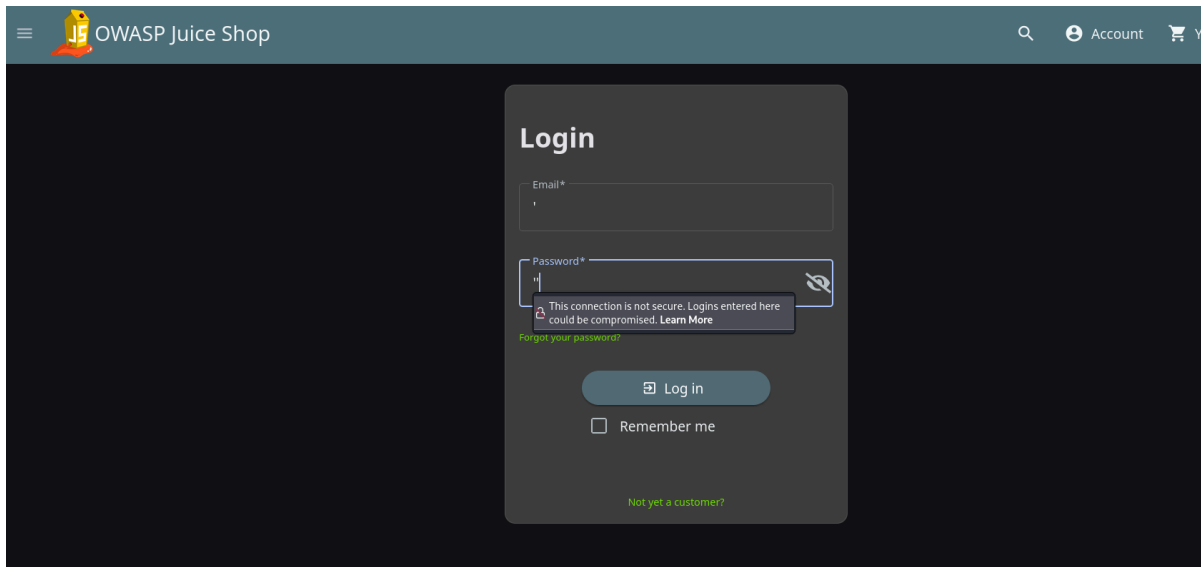


Figure 10. Evidence screenshot

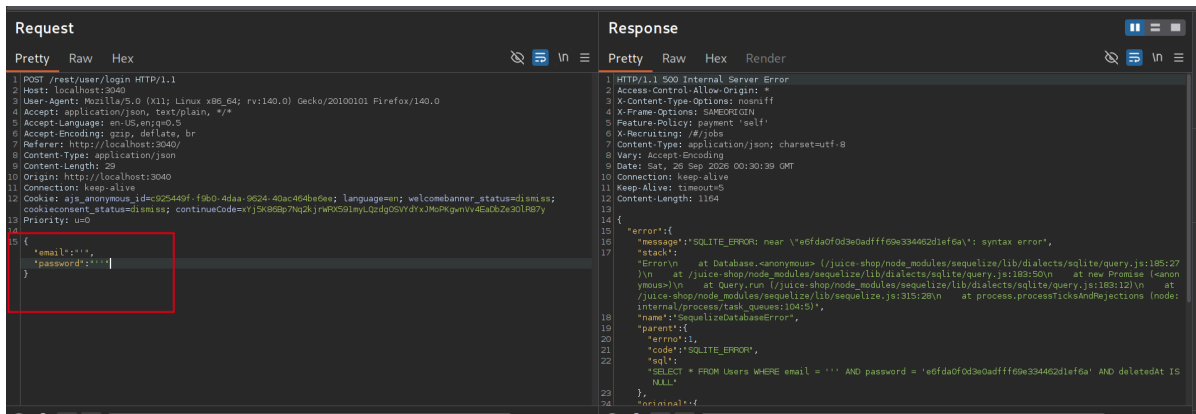


Figure 11. Evidence screenshot

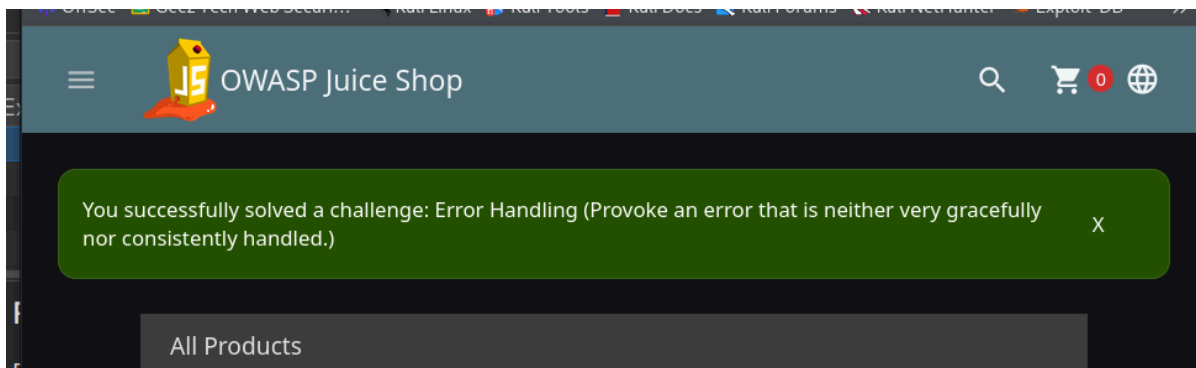


Figure 12. Evidence screenshot

Impact

- Information disclosure: error responses may expose database engine details, ORM information, SQL statements, stack traces, and internal file paths.
- Schema disclosure: the source report observed references to database tables and fields such as Users, email, password, and deletedAT.
- Attack facilitation: disclosed implementation details can make subsequent SQL injection and application attacks easier.
- The client-side presentation of unhandled objects indicates weak error handling and output processing.

Recommended Remediation

1. Return generic error messages to clients while retaining detailed diagnostics in protected server-side logs.

2. Use parameterized queries and robust exception handling.
3. Implement consistent error-handling middleware across API endpoints.
4. Disable debug mode, verbose stack traces, and development diagnostics in production.

F-06: SQL Injection in Product Search — Potential Full Database Compromise

Severity	Not specified in the source report
CVSS	Not specified
Affected Component	Product Search
Affected Endpoint	GET /rest/products/search?q=

Description

The product search endpoint was found to process attacker-controlled input in a way that permitted UNION-based SQL injection. The original testing demonstrated database schema enumeration and retrieval of user data through the search parameter.

Reproduction / Verification Steps

1. Open the product search function.
2. Submit a normal search and intercept the request in Burp Suite.
3. Modify the q parameter to a UNION-based SQL injection payload.
4. Enumerate database tables using: `qwert')) UNION SELECT name,2,3,4,5,6,7,8,sql FROM sqlite_master--`.
5. Use the discovered schema to retrieve user data with the tested payload:
`qwert'))+UNION+SELECT+id,username,email,password,role,'6','7','8','9'+FROM+Users--`.
6. Observe the returned user records.

Evidence

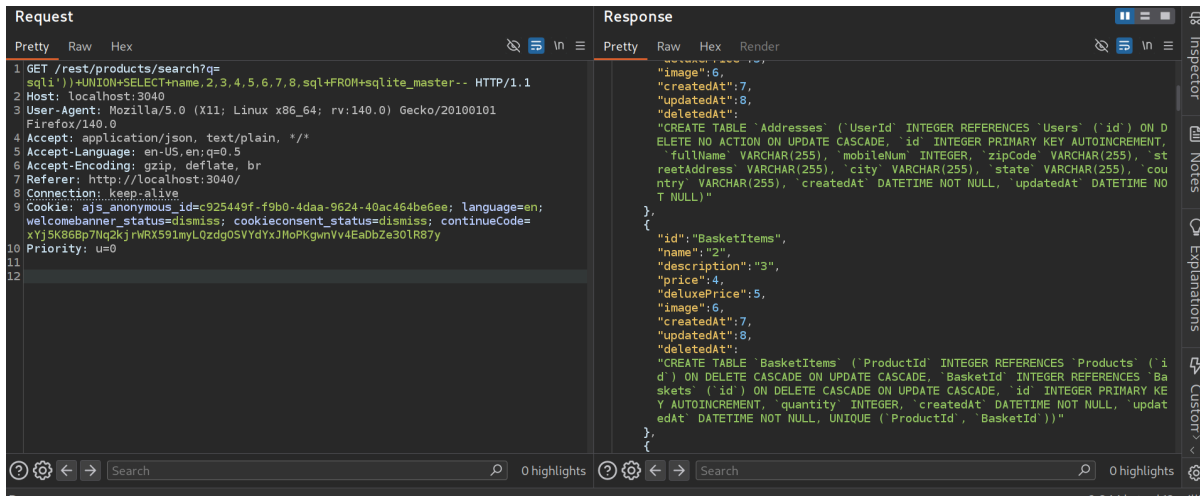


Figure 13. Evidence screenshot

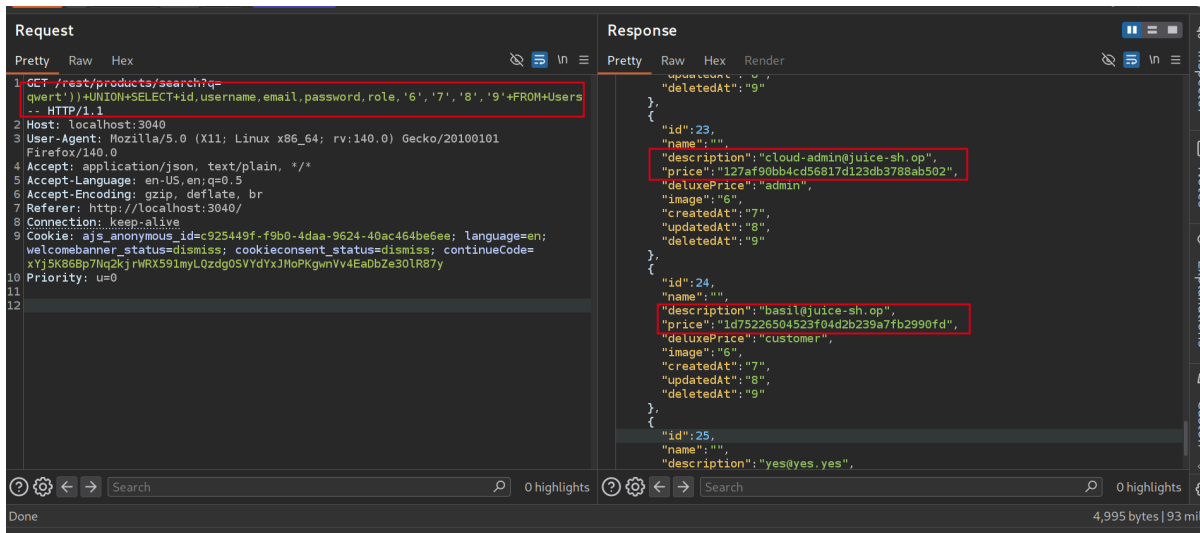


Figure 14. Evidence screenshot

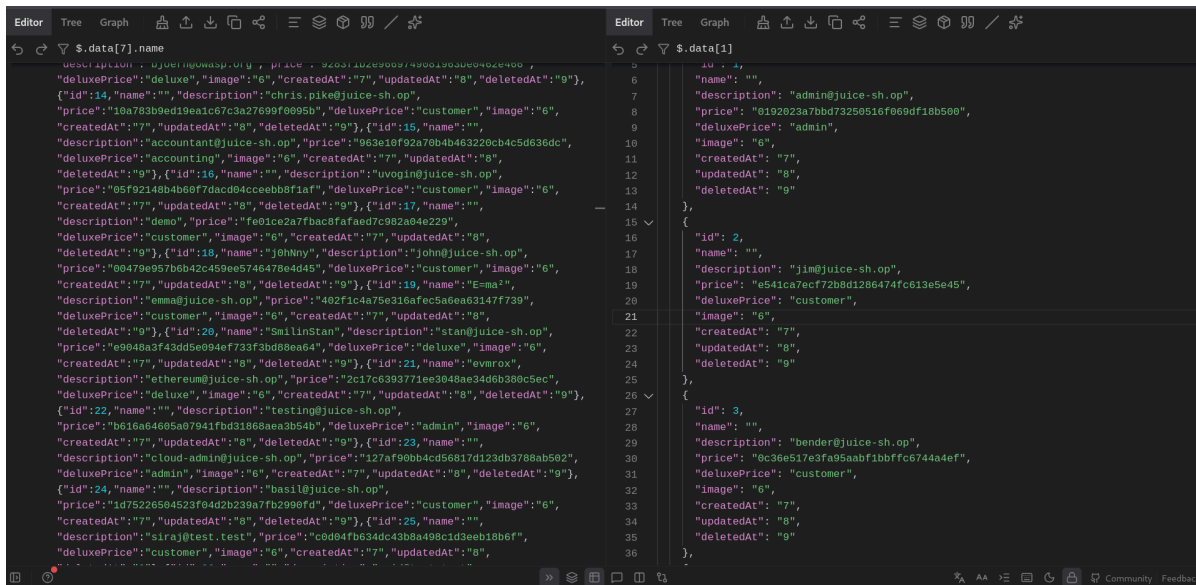


Figure 15. Evidence screenshot

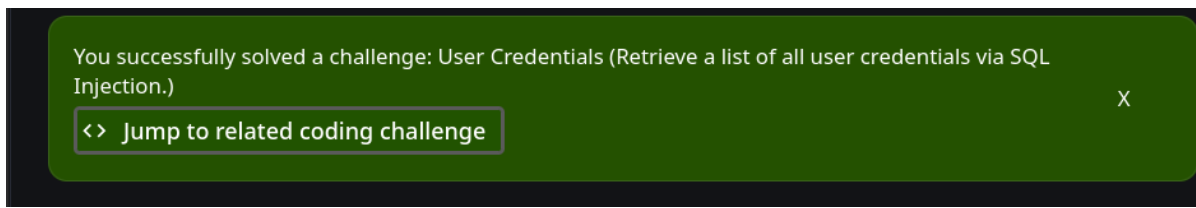


Figure 16. Evidence screenshot

Impact

- The source report states that credentials for 25 user accounts, including multiple administrators, were exposed.
- MD5 password hashes can be subjected to offline cracking.
- Successful exploitation can expose personal information, privilege-related data, and financial information.

Recommended Remediation

1. Use prepared statements or parameterized queries for the search endpoint.
2. Replace MD5 password storage with a dedicated password-hashing algorithm such as Argon2id or bcrypt.
3. Disable verbose SQL errors in production.

4. Apply least-privilege database permissions and consider WAF protections as defense in depth.

F-07: Unauthorized Role Assignment During Registration

Severity	Not specified in the source report
CVSS	Not specified
Affected Component	User Registration / Account Management
Affected Endpoint	POST /api/Users/ (UI: /#/register)

Description

The registration endpoint accepted a client-supplied role value. By modifying the registration request in Burp Suite and adding role: admin, the assessment created a new account that authenticated with administrative privileges.

Reproduction / Verification Steps

1. Open the registration page through the “Not yet a customer?” workflow.
2. Register a normal account with valid email, password, and security-question values.
3. Intercept the registration request in Burp Suite.
4. Send the request to Burp Repeater and add or modify the role field to admin.
5. Submit the modified request.
6. Log in with the newly created account and verify that it receives administrative privileges.

Evidence

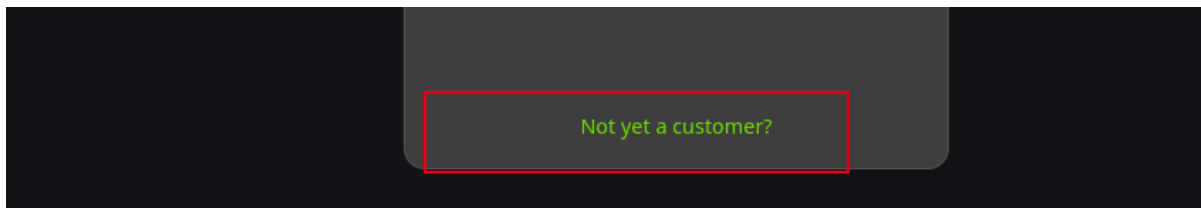


Figure 17. Evidence screenshot

BLACK-BOX PENETRATION TEST REPORT — OWASP JUICE SHOP

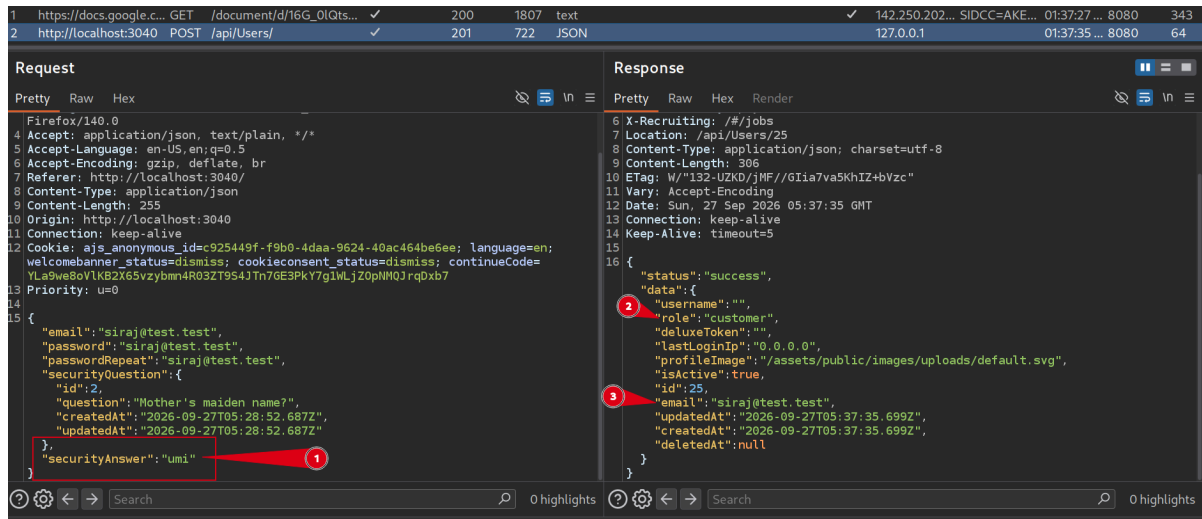


Figure 18. Evidence screenshot

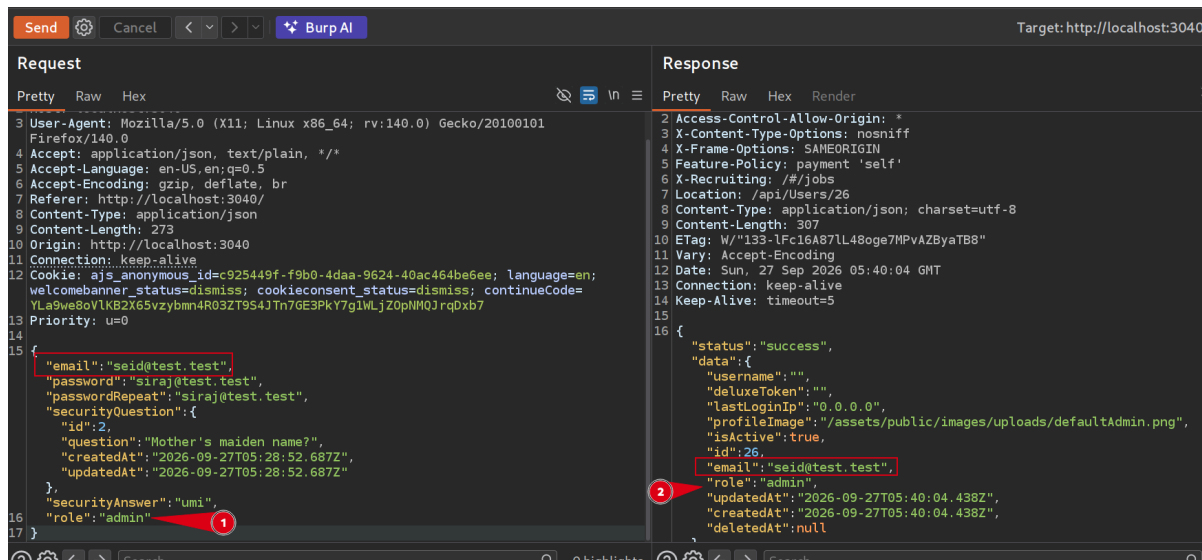


Figure 19. Evidence screenshot

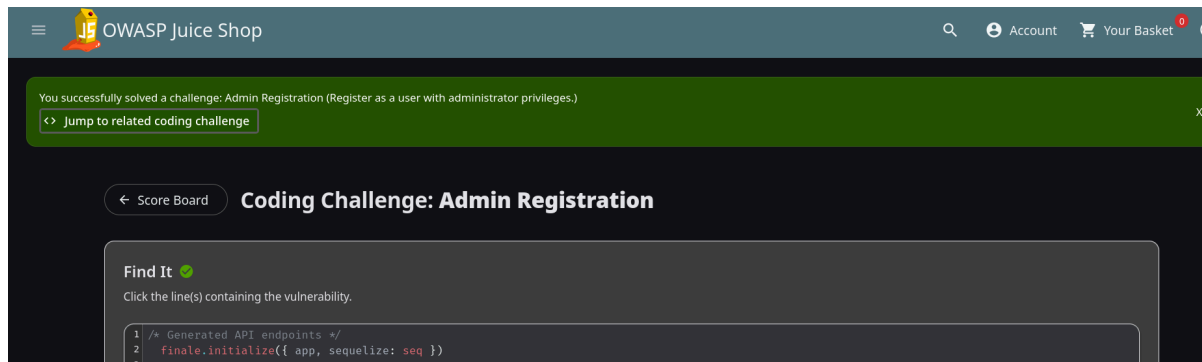


Figure 20. Evidence screenshot

Impact

- Any unauthenticated user may be able to self-assign an administrative role.
- Administrative privileges are granted through a normal login flow, producing a valid JWT with role:admin.
- The issue can lead to privilege escalation, unauthorized account management, and access to sensitive functionality.

Recommended Remediation

1. Ignore any role field supplied by the client during registration.
2. Assign the default customer role exclusively on the server.
3. Use an allow-list of accepted registration fields.
4. Enforce server-side authorization for every role change or privilege-elevation operation.
5. Add regression tests proving that registration cannot create privileged accounts.

F-08: Weak Administrative Password and Insufficient Brute-Force Protection

Severity	Not specified in the source report
CVSS	Not specified
Affected Component	Login / Authentication
Affected Endpoint	POST /rest/user/login (UI: /#/login)

Description

The assessment demonstrated that the administrator account accepted a password discovered through password-guessing activity using Burp Suite Intruder and a SecLists password wordlist. The authentication endpoint did not provide effective rate limiting, lockout, or progressive delay controls during the test period.

Reproduction / Verification Steps

1. Open the login page and enter the known administrator email with a test password.

2. Capture the request in Burp Suite and send it to Burp Intruder.
3. Mark the password value as the Intruder payload position.
4. Load the password wordlist and configure the attack.
5. Start the attack and monitor the responses.
6. Observe that a valid password can be identified without an effective server-side blocking mechanism.

Evidence

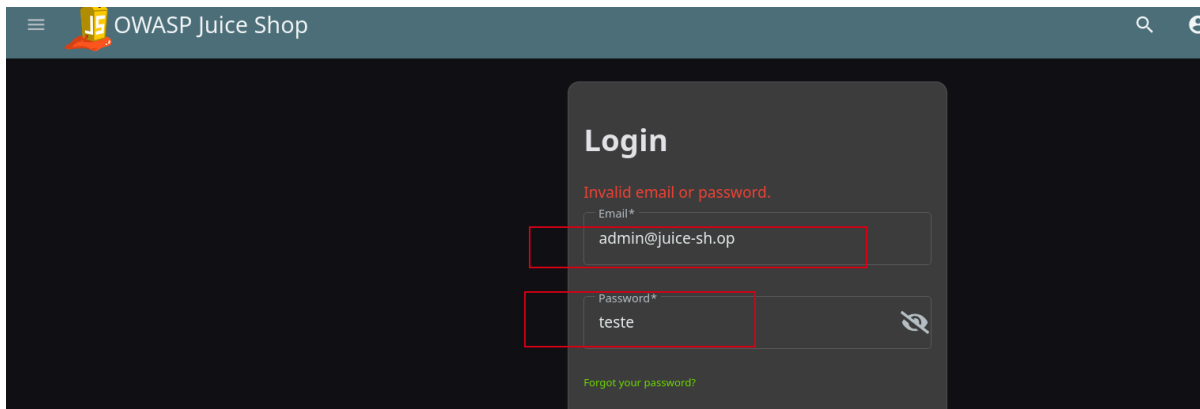


Figure 21. Evidence screenshot

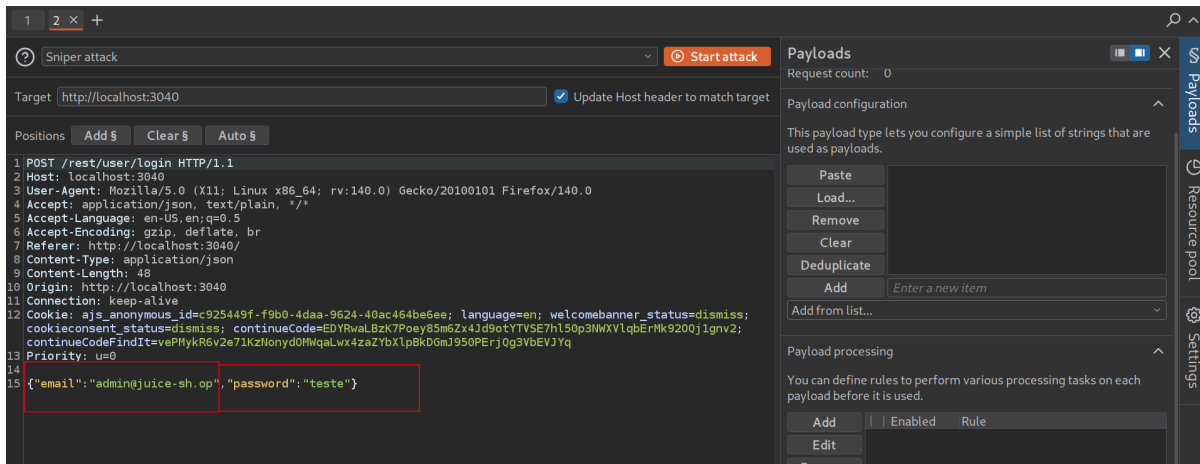


Figure 22. Evidence screenshot

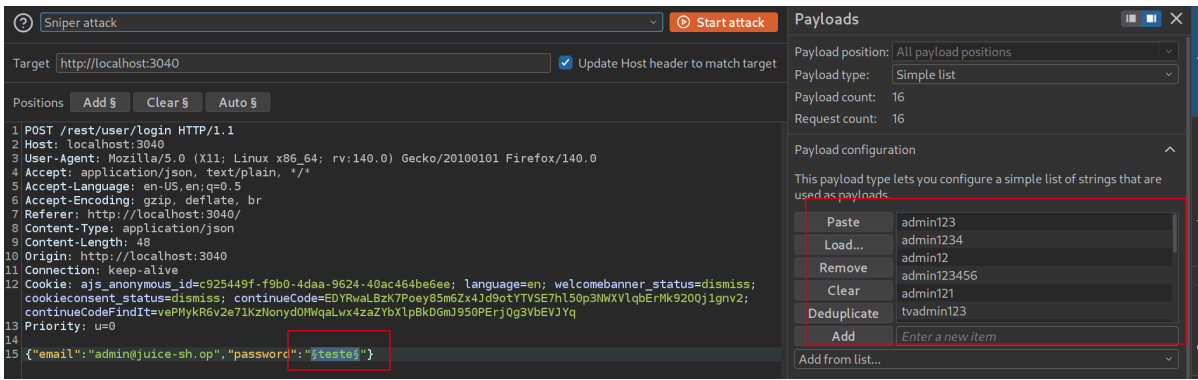


Figure 23. Evidence screenshot

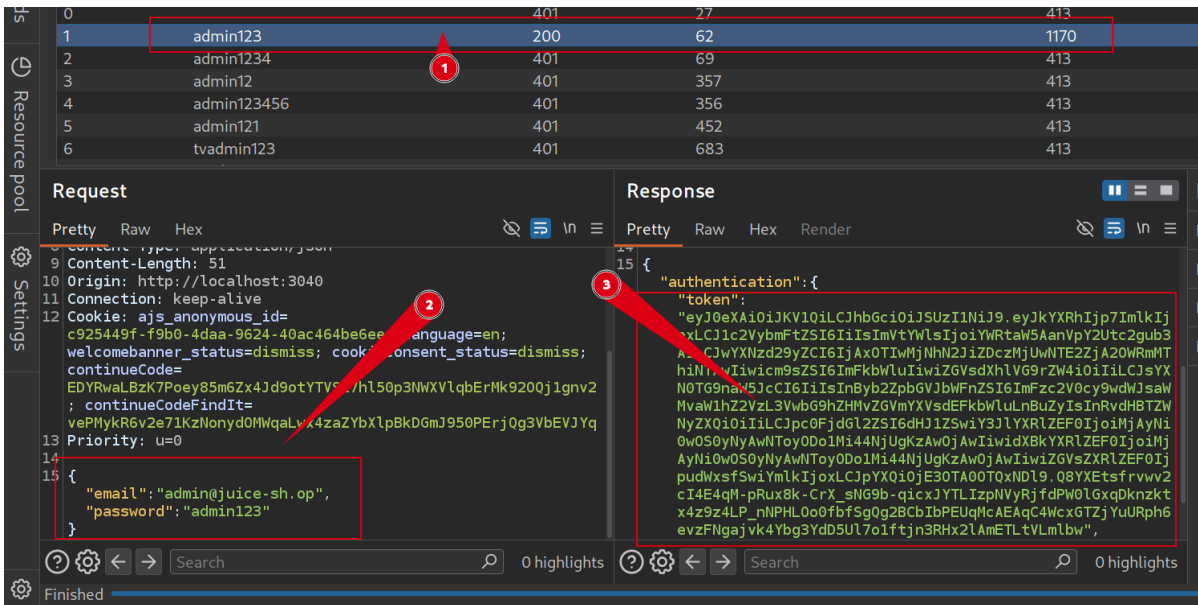


Figure 24. Evidence screenshot

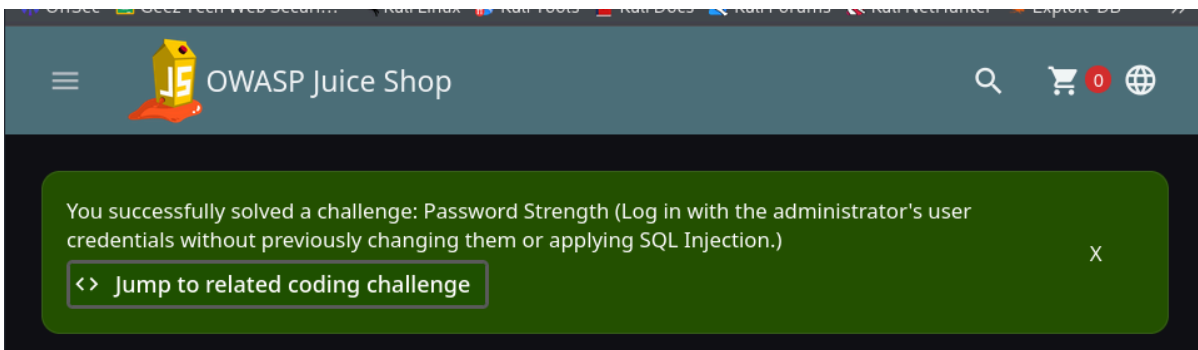


Figure 25. Evidence screenshot

Impact

- Repeated password-guessing attempts can be performed against administrator accounts.

- The absence of effective rate limiting or temporary blocking permits high-volume guessing.
- Successful discovery of a weak administrative password can result in administrative account compromise.

Recommended Remediation

1. Require long, unique administrative passwords and block commonly breached passwords.
2. Apply server-side rate limiting to the login endpoint.
3. Introduce progressive delays after repeated failed attempts.
4. Temporarily block or lock accounts or source identities after defined thresholds, with appropriate recovery controls.
5. Monitor and alert on abnormal authentication patterns.

F-09: Insufficient Authorization on Basket Access

Severity	Not specified in the source report
CVSS	Not specified
Affected Component	Basket / Authorization
Affected Endpoint	GET /rest/basket

Description

The basket API did not adequately bind a requested basket identifier to the authenticated user's identity. By changing the basket identifier in an intercepted request, the assessment accessed another user's basket.

Reproduction / Verification Steps

1. Log in with a normal account and open the basket.
2. Capture the basket request with Burp Suite.
3. Send the request to Burp Repeater.
4. Change the basket identifier to another value.
5. Repeat the test with another identifier to confirm that the behavior is not limited to a single record.

6. Observe that another basket can be retrieved.

Evidence

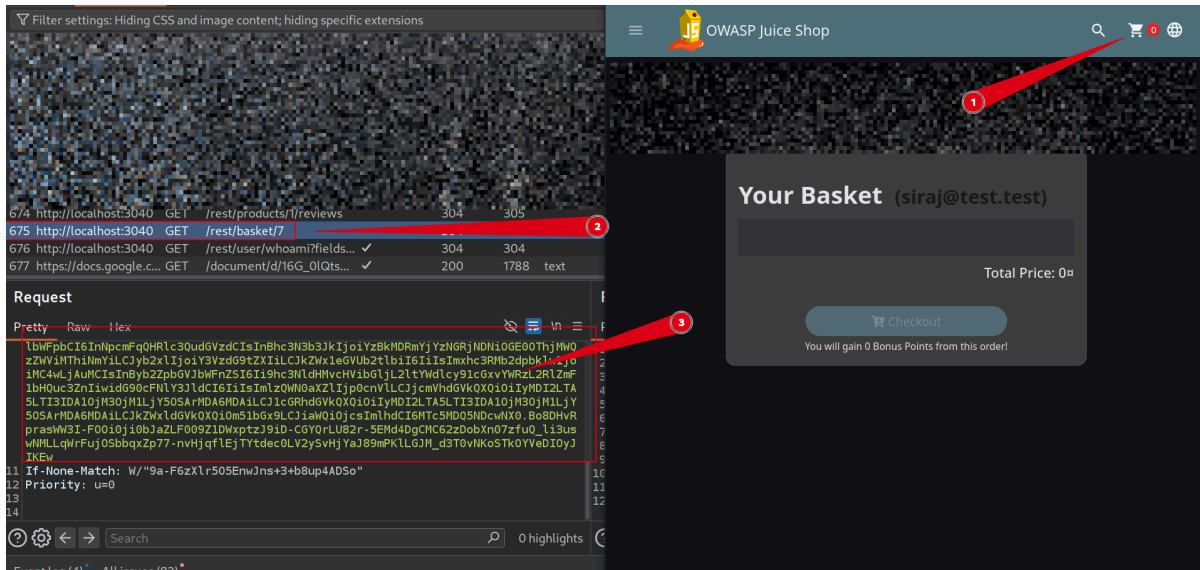


Figure 26. Evidence screenshot

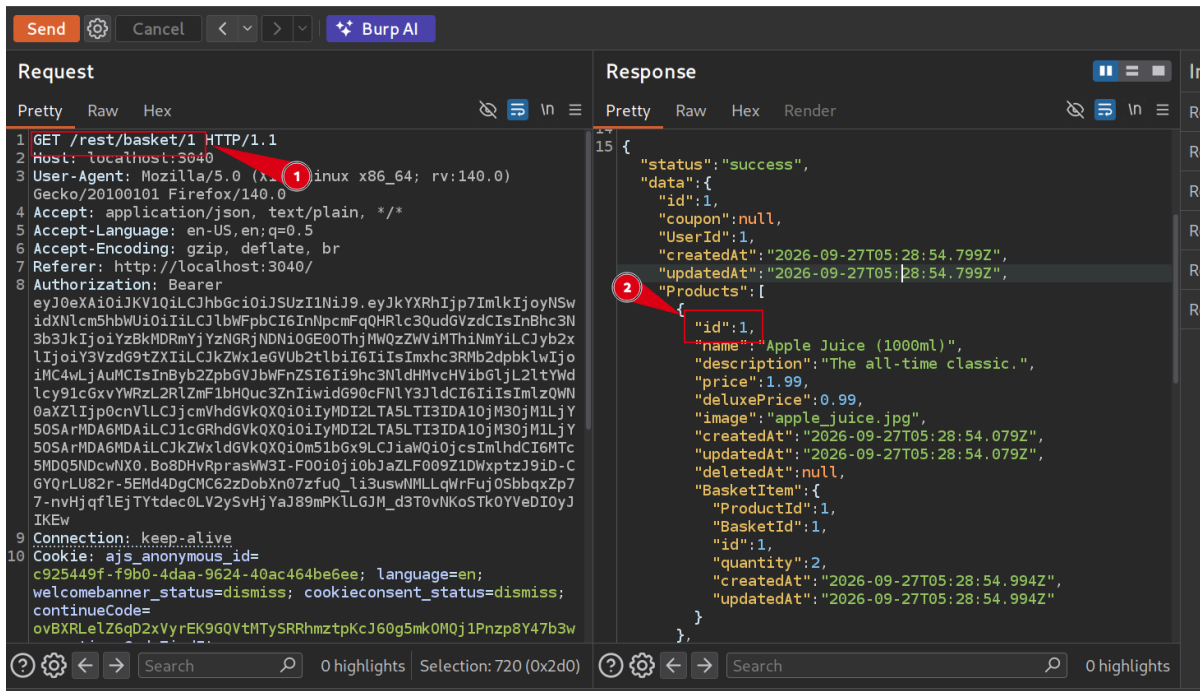


Figure 27. Evidence screenshot

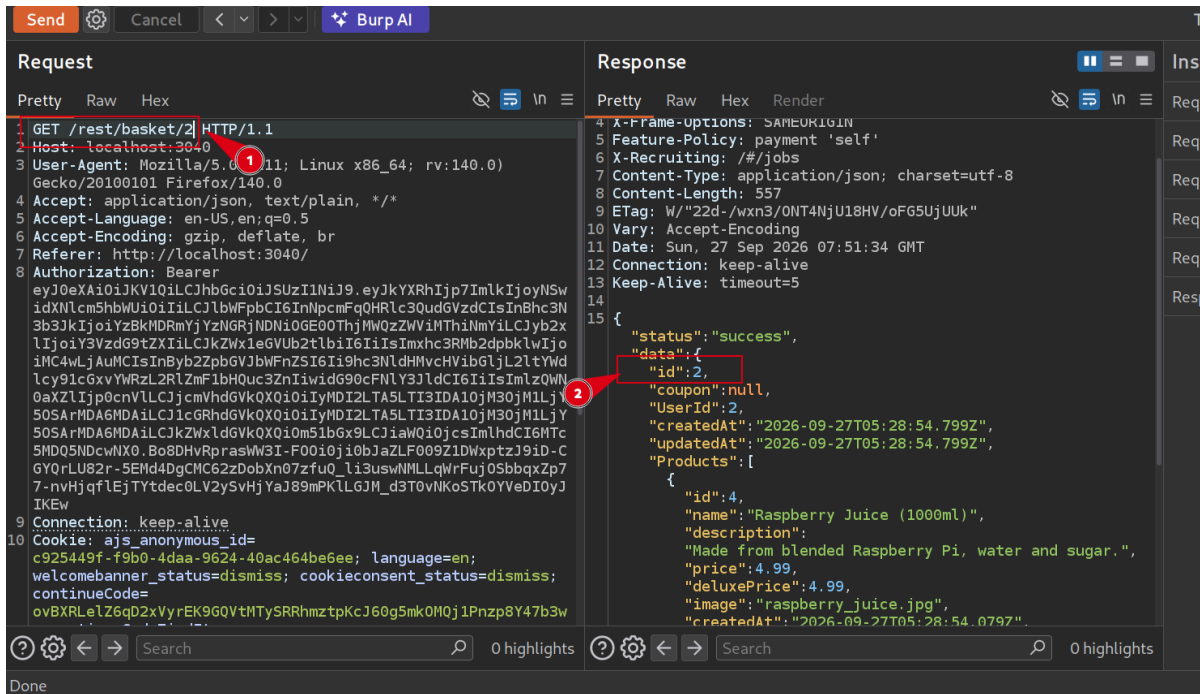


Figure 28. Evidence screenshot

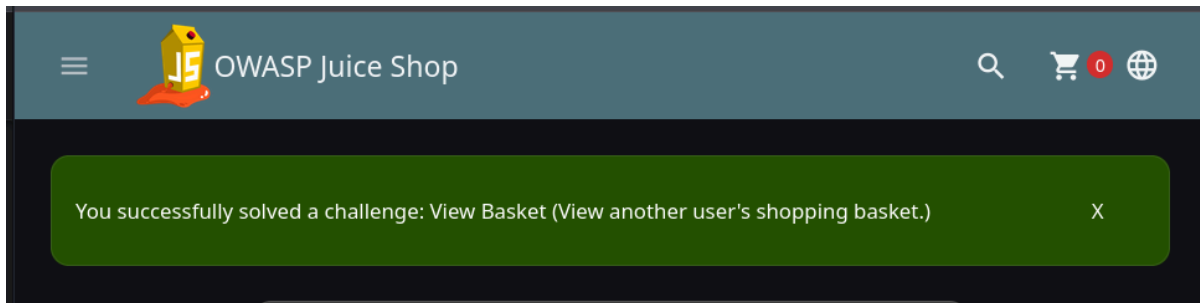


Figure 29. Evidence screenshot

Impact

- Authenticated users may read other users' baskets by changing the basketId value.
- Sequential identifiers make it possible to enumerate additional baskets.
- The response may also disclose the associated user identifier.

Recommended Remediation

1. Perform a server-side ownership check for every basket request.
2. Derive the basket from the authenticated session or JWT rather than trusting a client-supplied identifier.

3. Return 403 or 404 when a user attempts to access another user's resource.
4. Use non-sequential identifiers such as UUIDs as an additional defense against enumeration.

F-10: Privilege Escalation Through Improper JWT Signature Validation

Severity	Not specified in the source report
CVSS	Not specified
Affected Component	Authentication / JWT Authorization
Affected Endpoint	Administration-related protected endpoints (UI: /#/administration)

Description

The assessment found that JWT authorization could be manipulated by changing the role claim and using the none algorithm. The modified token was accepted by the application, demonstrating that token signature validation and authorization controls were insufficient.

Reproduction / Verification Steps

1. Register and authenticate as a normal customer account.
2. Identify the administration route; the source report notes that the route can be discovered in client-side code.
3. Modify the application route from /#/search to /#/administration.
4. Inspect the JWT and modify the role claim from customer to admin.
5. Change the JWT algorithm header to none and supply the modified token to the application.
6. Navigate to the administration page and verify access.

Evidence

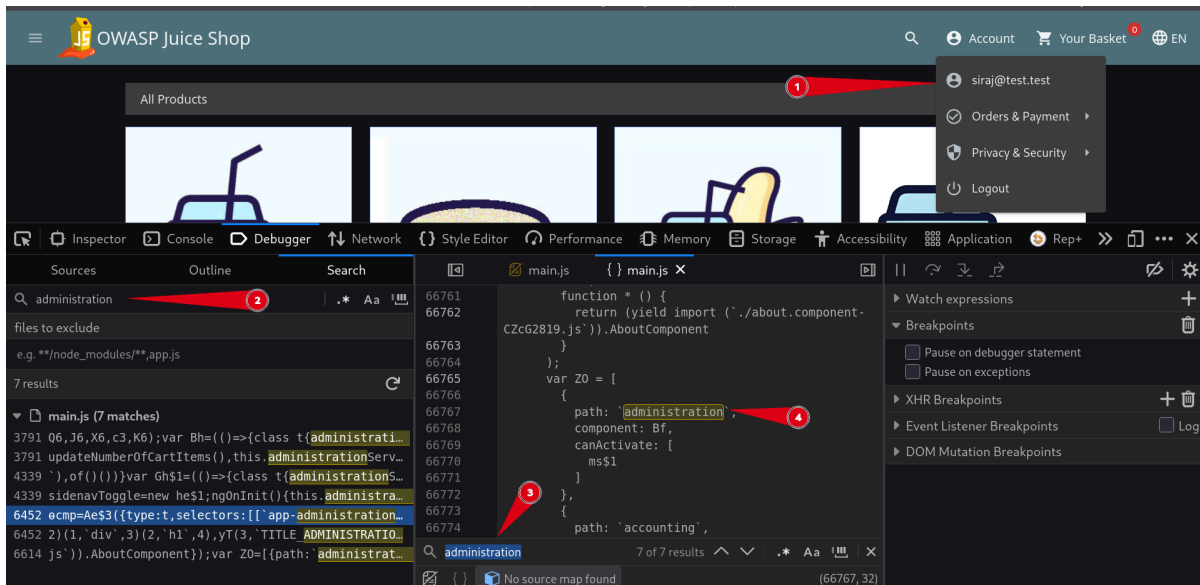


Figure 30. Evidence screenshot

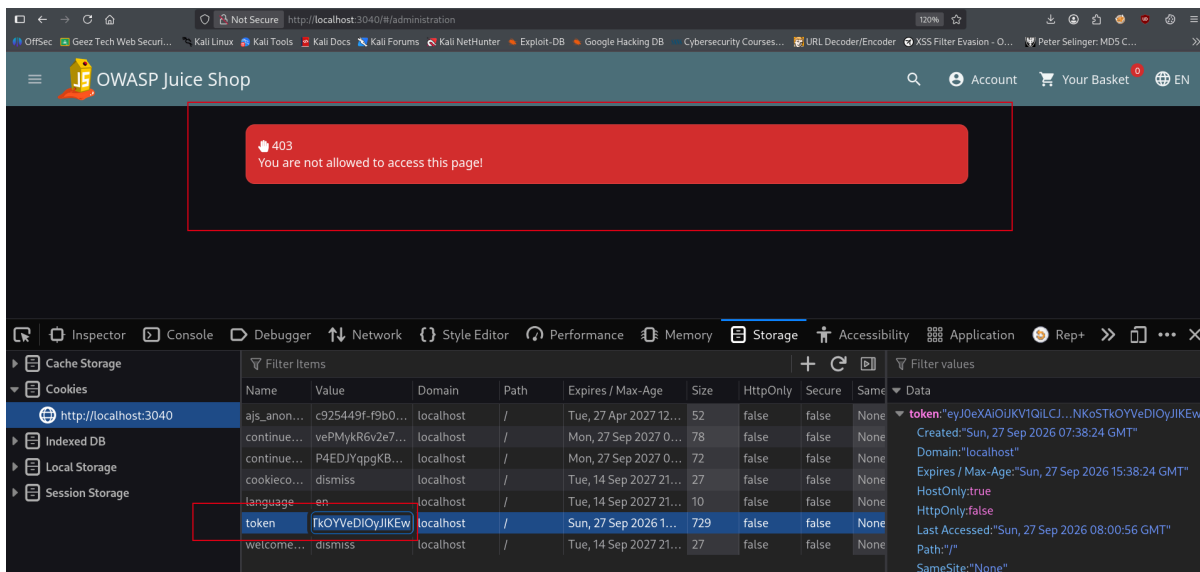


Figure 31. Evidence screenshot

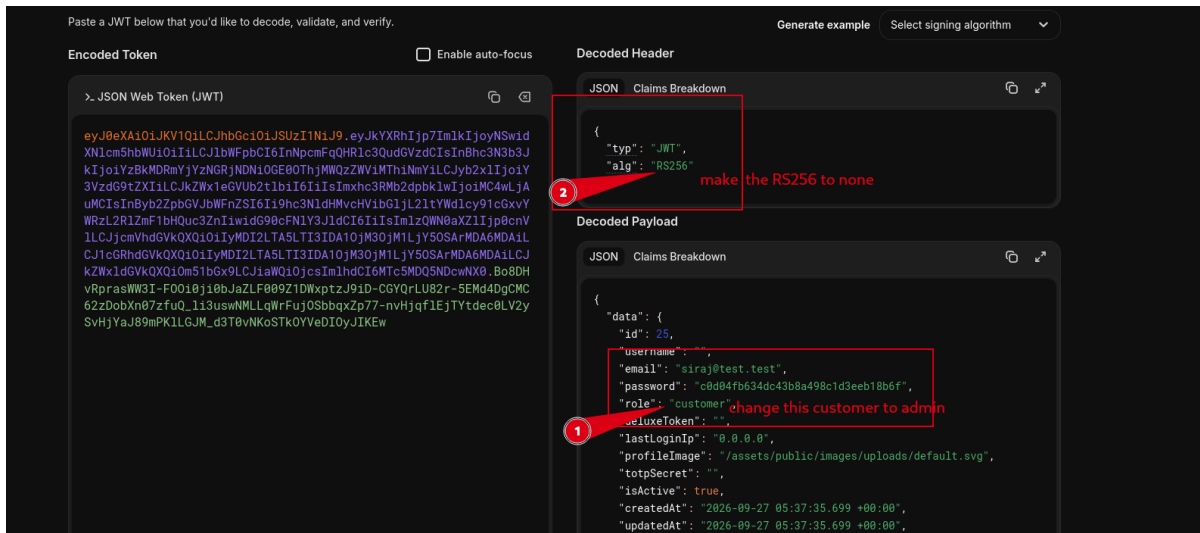


Figure 32. Evidence screenshot

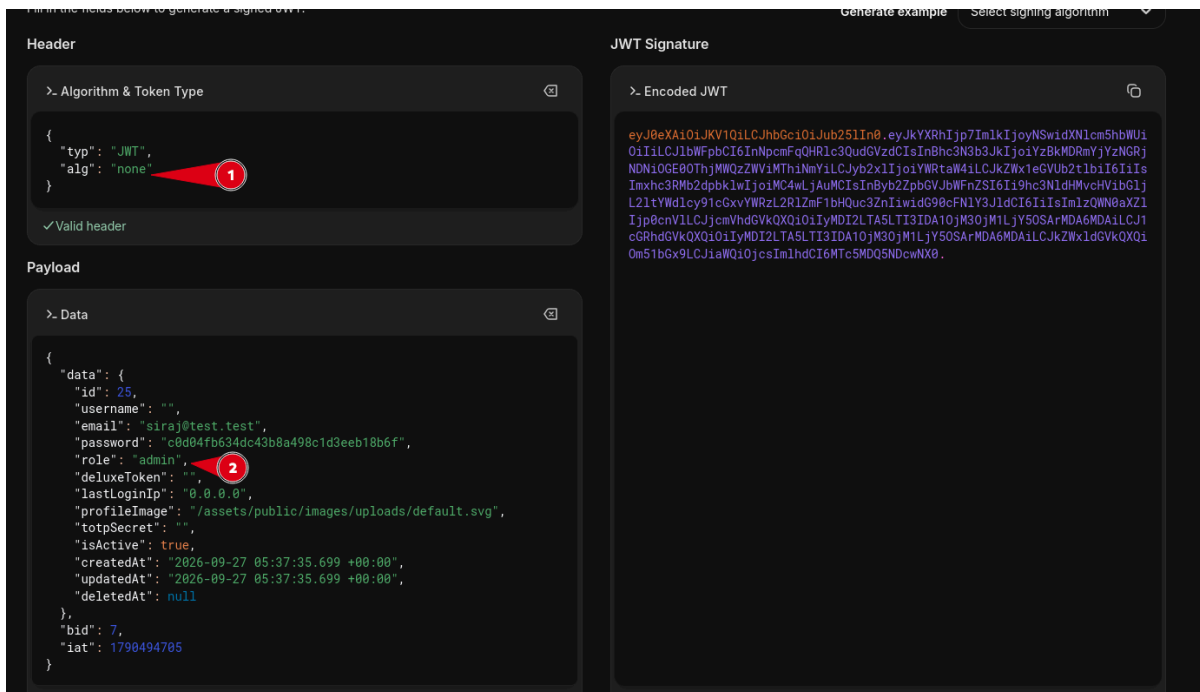


Figure 33. Evidence screenshot

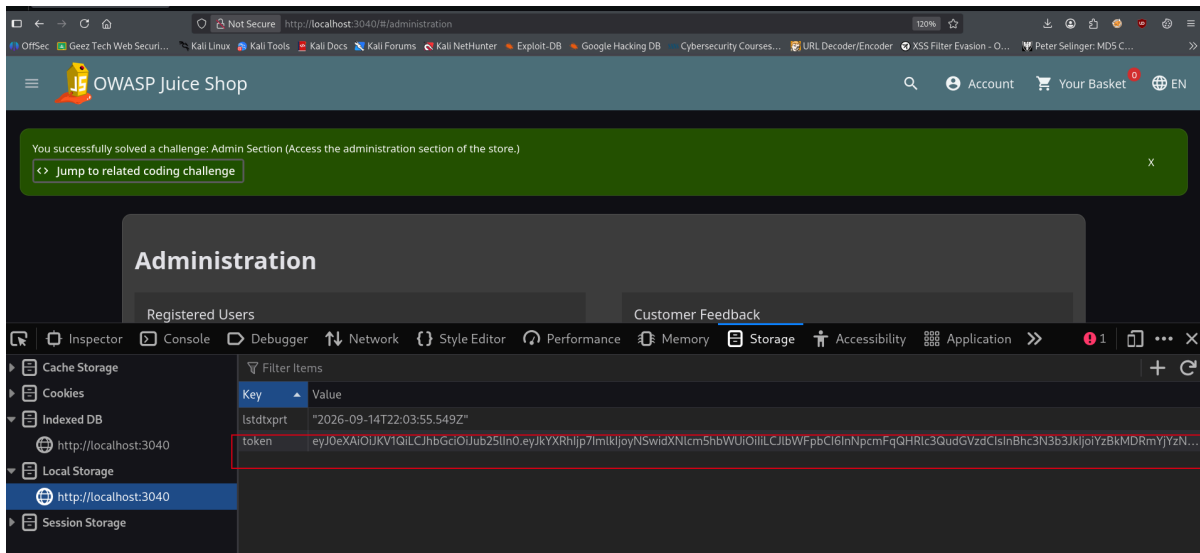


Figure 34. Evidence screenshot

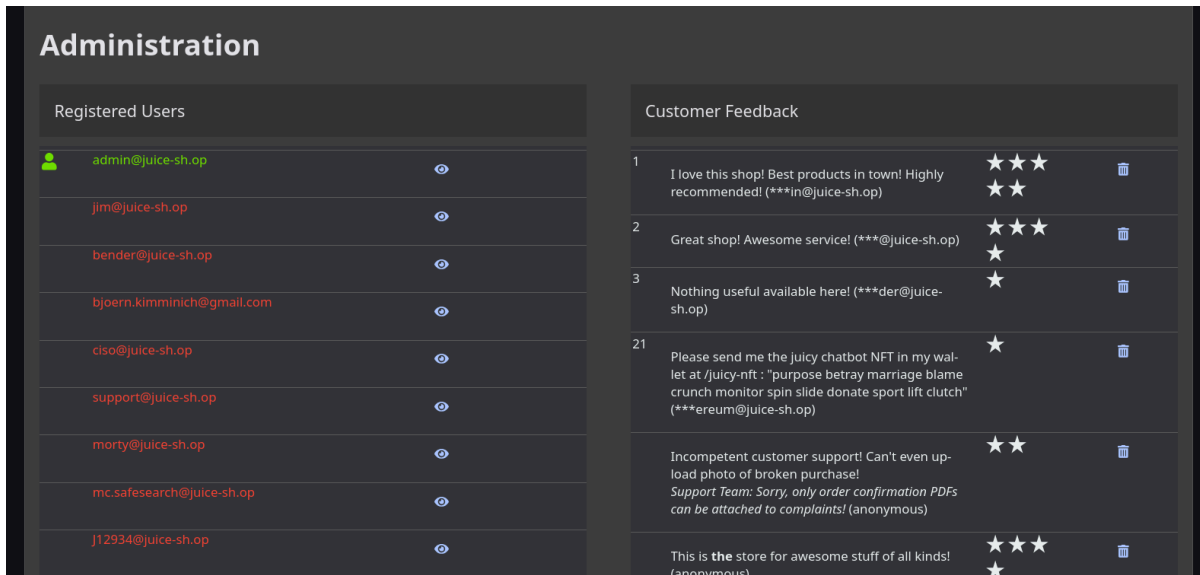


Figure 35. Evidence screenshot

Impact

- A normal user can potentially forge a token that claims administrative privileges.
- Successful exploitation provides access to administrative functionality.
- Acceptance of unsigned or improperly validated tokens undermines the application's authentication boundary.
- Hidden or hard-coded client-side routes do not provide meaningful authorization.

Recommended Remediation

1. Explicitly reject the none algorithm.
2. Use a strict allow-list of accepted JWT algorithms.
3. Verify the JWT signature on every protected request using the correct key.
4. Do not trust role claims supplied by the client without appropriate server-side authorization checks.
5. Apply server-side authorization middleware to all administrative endpoints.
6. Use a well-maintained JWT library with secure defaults and do not rely on hidden URLs.

F-11: Improper Authorization in Review Author Attribution

Severity	Not specified in the source report
CVSS	Not specified
Affected Component	Product Reviews / Author Attribution
Affected Endpoint	POST /rest/Product/{productId}/reviews (UI: /#/search)

Description

The product-review API accepted a client-supplied author value. By intercepting a legitimate review submission and replacing the author identifier with another user's email address, the assessment caused the review to be attributed to a different user.

Reproduction / Verification Steps

1. Authenticate with a normal account.
2. Create a product review through the normal interface.
3. Intercept the review request in Burp Suite.
4. Modify the author field to another user's email address, for example attacker@email.com.
5. Forward the modified request and verify that the review is accepted under the supplied identity.

Evidence

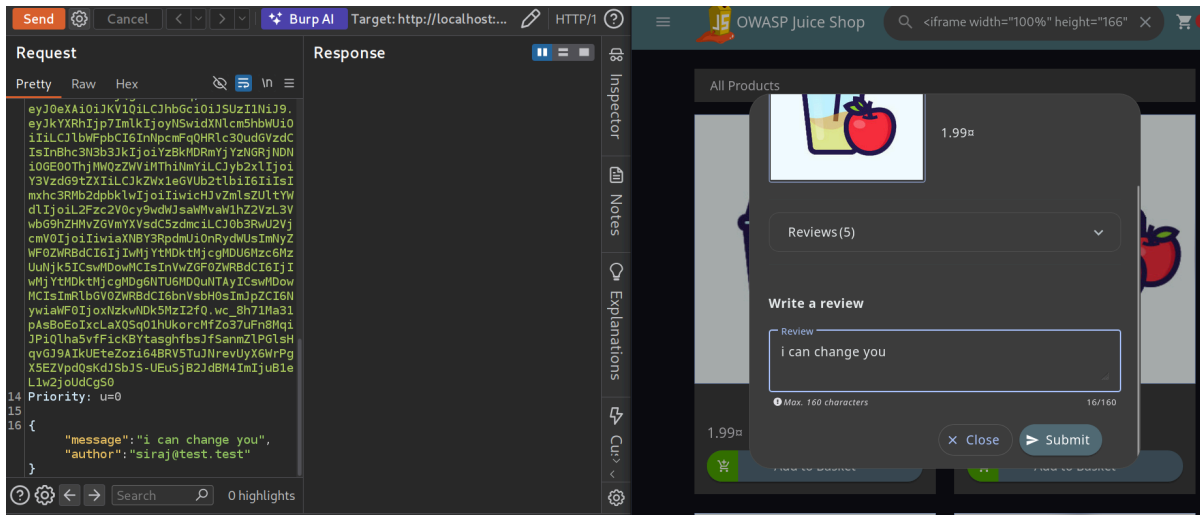


Figure 36. Evidence screenshot

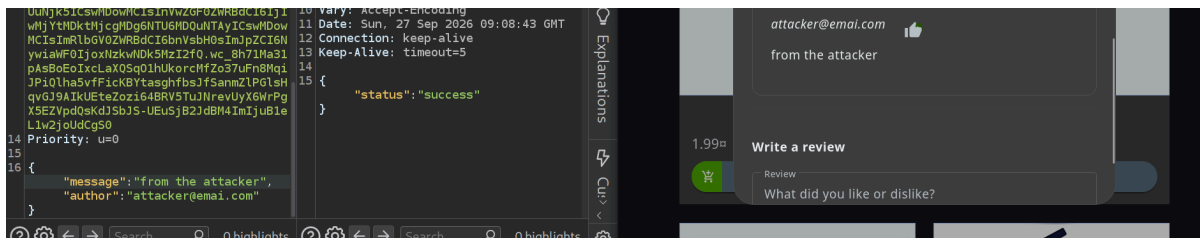


Figure 37. Evidence screenshot

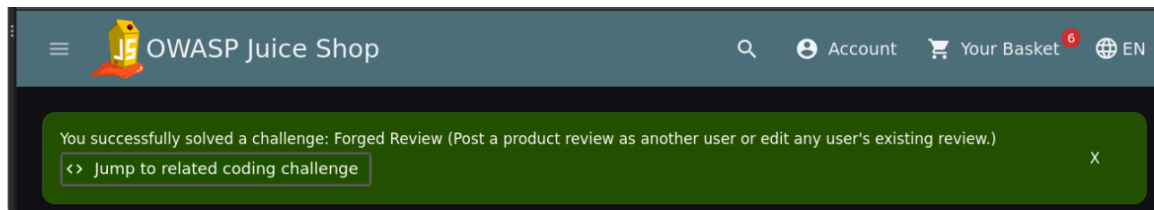


Figure 38. Evidence screenshot

Impact

- Attackers can submit content under another user's identity.
- Fake reviews can be attributed to legitimate users, creating impersonation and reputational risk.
- The application cannot reliably establish who authored a review when the author identity is client-controlled.

Recommended Remediation

1. Ignore client-supplied author identity fields.

2. Derive the author from the authenticated server-side session.
3. Enforce server-side authorization and ownership rules for review creation and modification.

F-12: Reflected XSS in Product Search

Severity	Not specified in the source report
CVSS	Not specified
Affected Component	Product Search
Affected Endpoint	GET /rest/products/search?q= (UI: /#/search)

Description

The product search functionality was observed to process crafted HTML/JavaScript content in a manner that resulted in JavaScript execution in the browser.

Reproduction / Verification Steps

1. Navigate to the product search functionality.
2. Submit the following payload through the search field: `<iframe src="javascript:alert('xss')">`.
3. Observe the application's response and the resulting JavaScript execution.

Evidence

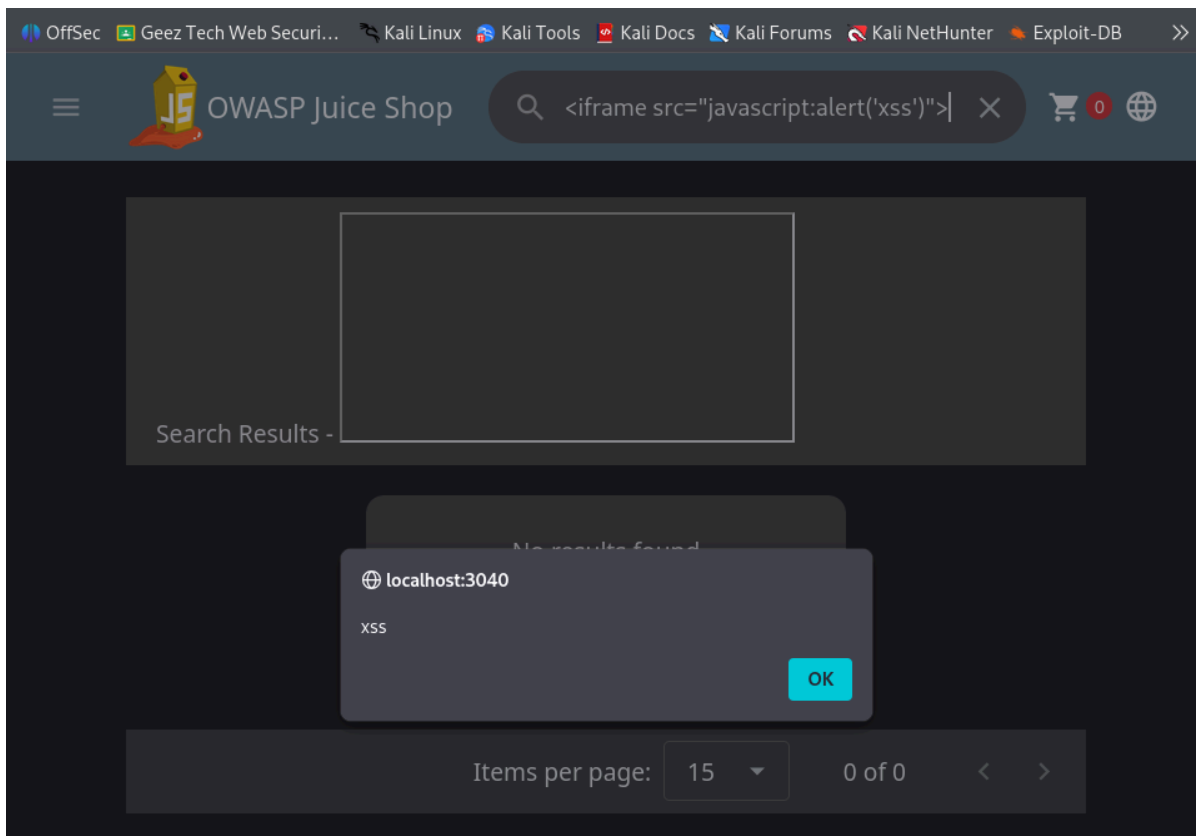


Figure 39. Evidence screenshot

Impact

- An attacker may execute JavaScript in the victim's browser.
- Depending on session protections, exploitation could support token or cookie theft and account compromise.
- An attacker may inject phishing content or redirect users to malicious destinations.

Recommended Remediation

1. Apply context-aware output encoding and avoid unsafe HTML insertion.
2. Sanitize user-controlled input on the server side as an additional control.
3. Implement a strict CSP to reduce the impact of successful injection.

F-13: Unauthenticated Exposure of Application Metrics

Severity	Low
CVSS	Not specified
Affected Component	Prometheus Metrics / Application Telemetry
Affected Endpoint	GET /metrics

Description

The application exposes a Prometheus-style metrics endpoint without authentication. The response contains internal application and runtime telemetry, including version information, challenge statistics, registered-user statistics, order information, wallet-related information, and other operational data.

Reproduction / Verification Steps

1. Perform directory discovery against the application. The source report used Gobuster against `http://localhost:3040/`.
2. Navigate to `http://localhost:3040/metrics`.
3. Observe that the endpoint returns application and runtime metrics without requiring authentication.

Evidence

```

~: gobuster
> gobuster dir -u http://127.0.0.1:3040/ \
-w /usr/share/seclists/Discovery/Web-Content/DirBuster-2007_directory-list-2.3-medium.txt \
--exclude-length 9393
=====
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
[+] Url: http://127.0.0.1:3040/
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/seclists/Discovery/Web-Content/DirBuster-2007_directory-list-2.3-medium.txt
[+] Negative Status codes: 404
[+] Exclude Length: 9393
[+] User Agent: gobuster/3.8.2
[+] Timeout: 10s
=====
Starting gobuster in directory enumeration mode
=====
media (Status: 301) [Size: 155] [--> /media/]
profile (Status: 500) [Size: 1043]
video (Status: 200) [Size: 2232676]
assets (Status: 301) [Size: 156] [--> /assets/]
redirect (Status: 500) [Size: 2531]
ftp (Status: 200) [Size: 11307]
api (Status: 500) [Size: 2408]
Video (Status: 200) [Size: 2232676]
restaurants (Status: 500) [Size: 2424]
infrastructure (Status: 200) [Size: 9642]
promotion (Status: 200) [Size: 5863]
Profile (Status: 500) [Size: 1043]
FTP (Status: 200) [Size: 11307]
rest (Status: 500) [Size: 2410]
restricted (Status: 500) [Size: 2422]
Redirect (Status: 500) [Size: 2531]
metrics (Status: 200) [Size: 26330]
Progress: 8522 / 220558 (3.86%)

```

Figure 40. Evidence screenshot

```

localhost:3040/metrics x +
Not Secure http://localhost:3040/metrics
# HELP juiceshop challenges progress Number of coding challenges grouped by progression phase.
# TYPE juiceshop challenges progress gauge
juiceshop_challenges_progress{phase="find it",app="juiceshop"} 1
juiceshop_challenges_progress{phase="fix it",app="juiceshop"} 0
juiceshop_challenges_progress{phase="unsolved",app="juiceshop"} 34
# HELP juiceshop cheat_score Overall probability that any hacking or coding challenges were solved by cheating.
# TYPE juiceshop cheat_score gauge
juiceshop_cheat_score{app="juiceshop"} 0
# HELP juiceshop coding_challenges accuracy Overall accuracy while solving coding challenges grouped by phase.
# TYPE juiceshop coding_challenges accuracy gauge
juiceshop_coding_challenges_accuracy{phase="find it",app="juiceshop"} Nan
juiceshop_coding_challenges_accuracy{phase="fix it",app="juiceshop"} Nan
# HELP juiceshop orders_placed_total Number of orders placed in OWASP Juice Shop.
# TYPE juiceshop orders_placed_total gauge
juiceshop_orders_placed_total{app="juiceshop"} 3
# HELP juiceshop users_registered Number of registered users grouped by customer type.
# TYPE juiceshop users_registered gauge
juiceshop_users_registered{type="standard",app="juiceshop"} 10
juiceshop_users_registered{type="deluxe",app="juiceshop"} 4
# HELP juiceshop users_registered_total Total number of registered users.
# TYPE juiceshop users_registered_total gauge
juiceshop_users_registered_total{app="juiceshop"} 23
# HELP juiceshop wallet_balance_total Total balance of all users' digital wallets.
# TYPE juiceshop wallet_balance_total gauge
juiceshop_wallet_balance_total{app="juiceshop"} 13887
# HELP juiceshop user_social_interactions Number of social interactions with users grouped by type.
# TYPE juiceshop user_social_interactions gauge
juiceshop_user_social_interactions{type="review",app="juiceshop"} 29
juiceshop_user_social_interactions{type="feedback",app="juiceshop"} 8
juiceshop_user_social_interactions{type="complaint",app="juiceshop"} 1

```

Figure 41. Evidence screenshot

Impact

- Unauthenticated users can retrieve internal telemetry.
- Version information and runtime details assist reconnaissance.
- Challenge and usage statistics reveal aspects of the application's internal state.
- User-registration and application-usage statistics provide additional intelligence to an attacker.

Recommended Remediation

1. If metrics are not intended for public access, require authentication or restrict the endpoint to trusted monitoring systems.
2. Place monitoring endpoints on a management network or behind an access-control layer.
3. Minimize sensitive information included in publicly reachable telemetry.

F-14: Hardcoded Credentials Exposed in Client-Side JavaScript

Severity	Not specified in the source report
CVSS	Not specified
Affected Component	Client-side JavaScript / Authentication Logic
Affected Endpoint	Client-side JavaScript bundle (e.g., app.js/main.js)

Description

Credentials for a testing account were identified directly in client-side JavaScript. Because browser-delivered JavaScript is available to users, hardcoded credentials in the client bundle can be extracted without privileged access.

Reproduction / Verification Steps

1. Open a product page and inspect the application using browser Developer Tools.
2. Open the Sources/Debugger view.
3. Search the client-side JavaScript for the relevant test credential string.
4. Use the discovered credentials to authenticate to the application.
5. Verify that the credentials provide access to the associated account.

Evidence

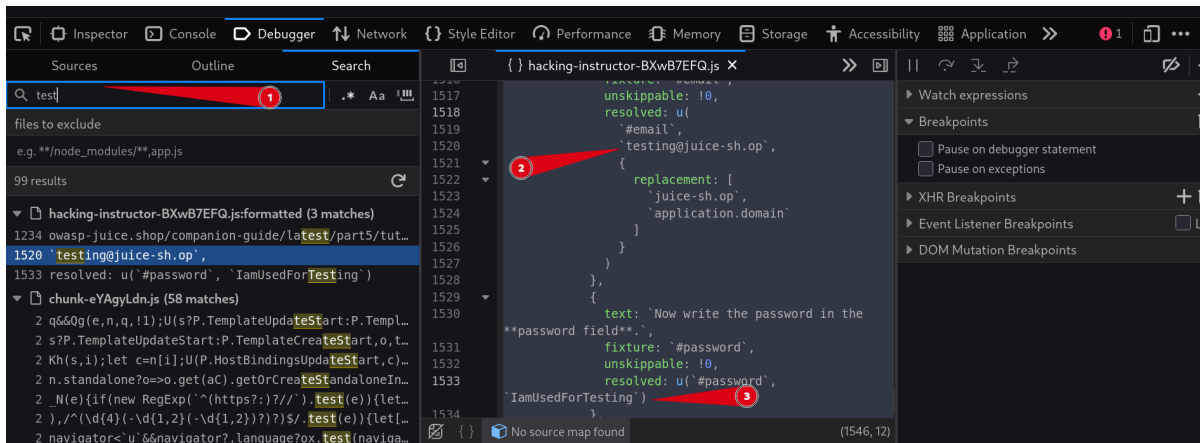


Figure 42. Evidence screenshot

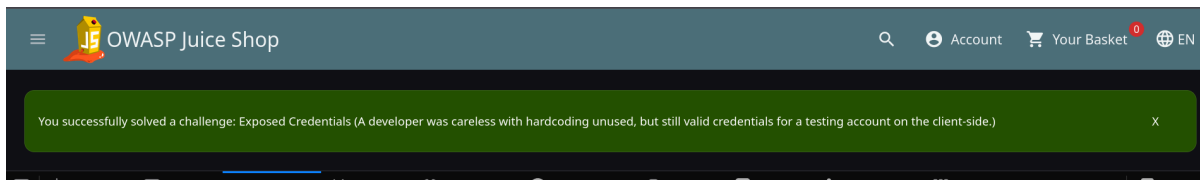


Figure 43. Evidence screenshot

Impact

- Valid credentials can be extracted directly from publicly delivered code.
- An attacker may use the credentials to access user data, order information, or application functionality.
- If the account is reused elsewhere, the exposure may facilitate further compromise.

Recommended Remediation

1. Never embed passwords, API keys, or other secrets in client-side code.
2. Move sensitive configuration and credentials to server-side environment variables or a secret-management system.
3. Remove or disable unused testing accounts before production deployment.
4. Rotate any credentials that have already been exposed.

F-15: Credential Exposure Through Publicly Available OSINT and Weak Password Construction

Severity	Not specified in the source report
CVSS	Not specified
Affected Component	User Authentication
Affected Endpoint	POST /rest/user/login

Description

The assessment demonstrated that the MC SafeSearch account used a password derived from publicly available information. The source report states that the user's email was exposed through application content and that a publicly available video revealed the password pattern based on the user's dog's name with simple character substitutions.

Reproduction / Verification Steps

1. Identify the user's email address: mc.safesearch@juice-sh.op.
2. Use publicly available information referenced by the challenge to identify the password pattern.
3. Construct the tested credentials: email mc.safesearch@juice-sh.op and password Mr. N00dles.
4. Submit the credentials through the login page.
5. Observe that authentication succeeds.

Evidence

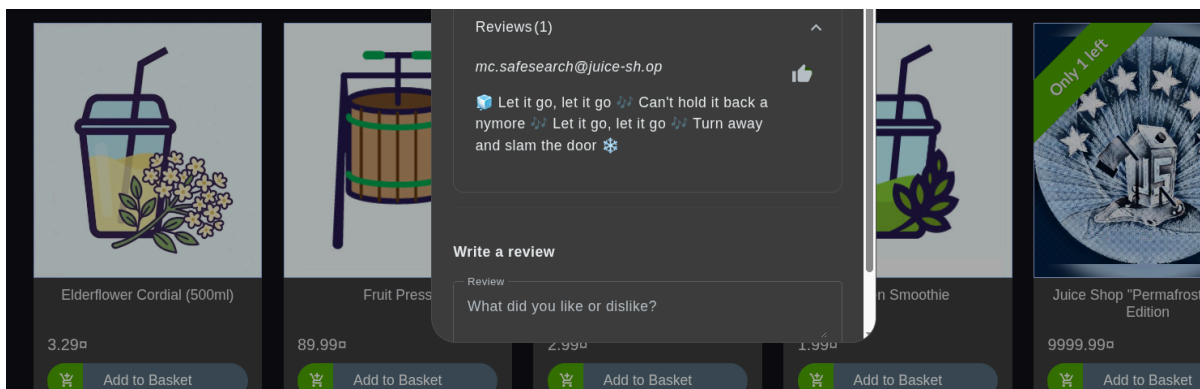


Figure 44. Evidence screenshot

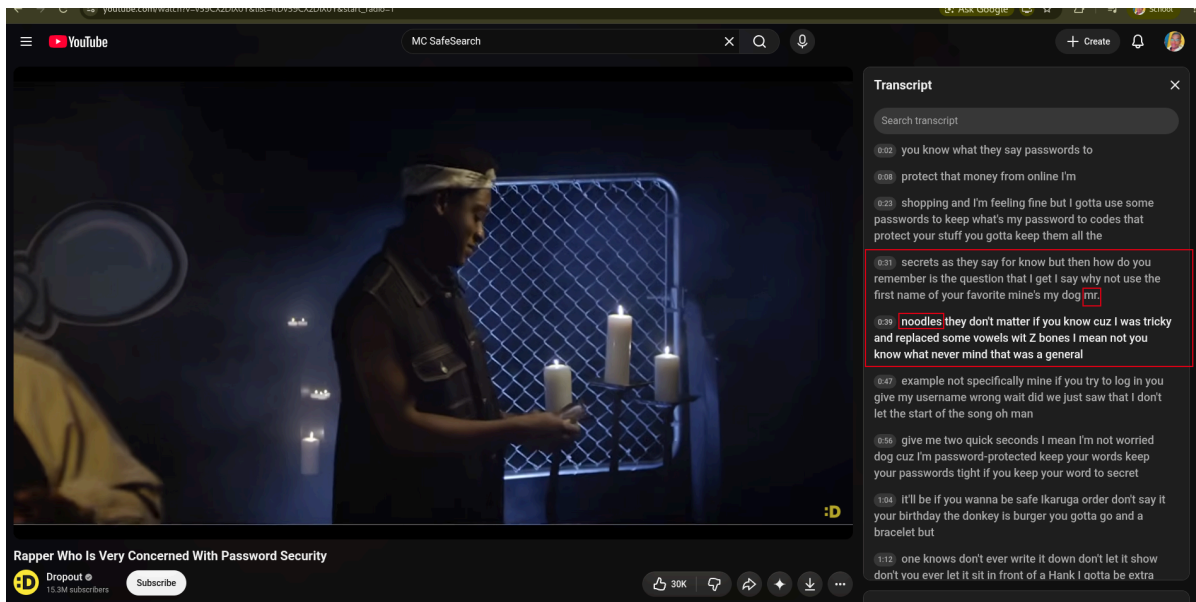


Figure 45. Evidence screenshot

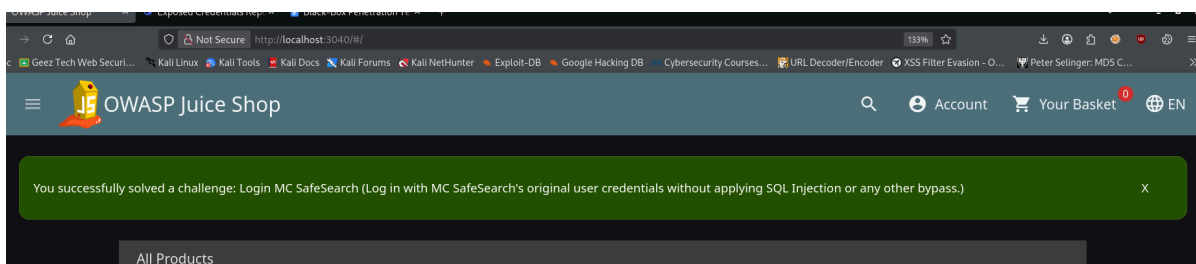


Figure 46. Evidence screenshot

Impact

- Publicly available information can be combined with weak password construction to compromise an account.
- The impact increases if the compromised account has access to sensitive data or privileged functions.
- The finding demonstrates that simple character substitutions do not provide adequate protection against targeted password guessing.

Recommended Remediation

1. Enforce strong password policies and block predictable or commonly breached passwords.
2. Reduce unnecessary exposure of user email addresses and other account information.

3. Implement MFA for accounts where practical.
4. Apply rate limiting and temporary account protections to reduce password-guessing attacks.

F-16: Forgotten Developer Backup File Exposed

Severity	Not specified in the source report
CVSS	Not specified
Affected Component	Backup Files / Web Server
Affected Endpoint	GET /ftp/package.json.bak (or similar backup paths)

Description

A developer backup file was accessible through the public FTP directory. Backup files such as .bak, .old, .swp, or .zip can contain source code, dependency information, configuration, or secrets and should not be deployed into web-accessible paths.

Reproduction / Verification Steps

1. Navigate to the application and locate the /ftp directory.
2. Identify the package.json.bak file.
3. Attempt to access or download the backup file.
4. If direct access is blocked, the original assessment describes testing URL-encoding or related path-handling behavior to determine whether the file can still be served.
5. Review the retrieved file for dependencies and configuration information.

Evidence

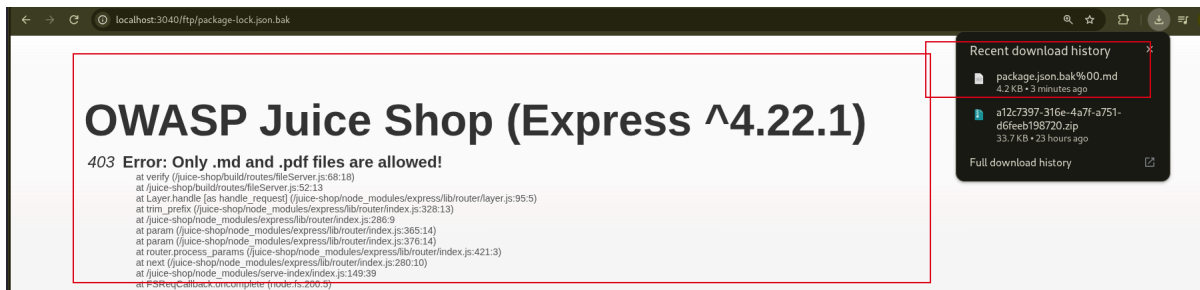


Figure 47. Evidence screenshot

```
1 {
2   "name": "juice-shop",
3   "version": "6.2.0-SNAPSHOT",
4   "description": "An intentionally insecure
5   JavaScript Web Application",
6   "homepage": "http://owasp-juice.shop",
7   "author": "Björn Kimminich <bjoern.kimminich@
8   owasp.org> (https://kimminich.de)",
9   "contributors": [
10    "Björn Kimminich",
11    "Jannik Hollenbach",
12    "Aashish683",
13    "greenkeeper[bot]",
14    "MarcRler",
15    "agrawalarpit14",
16    "Scar26",
```

Figure 48. Evidence screenshot

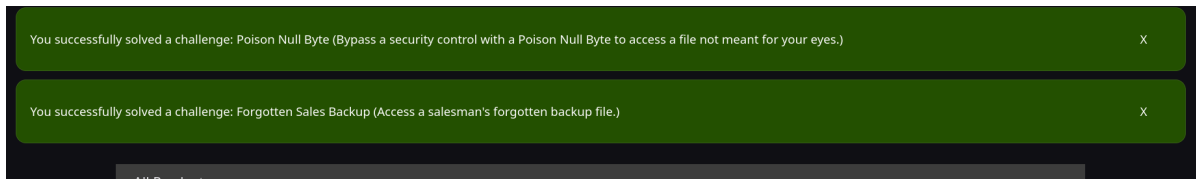


Figure 49. Evidence screenshot

Impact

- Attackers may obtain application source or dependency information.
- Dependency versions can be used to identify known vulnerabilities.
- Backup files may contain hardcoded secrets, API keys, or other deployment information.

Recommended Remediation

1. Ensure backup files are not deployed to production web roots.
2. Configure the web server to deny access to backup and hidden-file extensions.
3. Add CI/CD checks that fail deployments containing unintended backup artifacts.
4. Store backups outside the public web root.

F-17: Leaked API Key in Client-Side Code

Severity	Not specified in the source report
CVSS	Not specified
Affected Component	Client-side JavaScript / Public Configuration
Affected Endpoint	Client-side JavaScript files such as main.js

Description

The assessment identified the possibility of an API key or secret being embedded in client-side JavaScript. Client-delivered source code is publicly inspectable, so secrets stored there should be treated as compromised.

Reproduction / Verification Steps

1. Open the application and Developer Tools.
2. Navigate to the Sources view.
3. Search JavaScript bundles for keywords such as key, api, token, secret, or google.
4. Identify the exposed key value.
5. The source report describes submitting the discovered key through the application's Contact Us or Customer Feedback function as part of the challenge verification.

Evidence

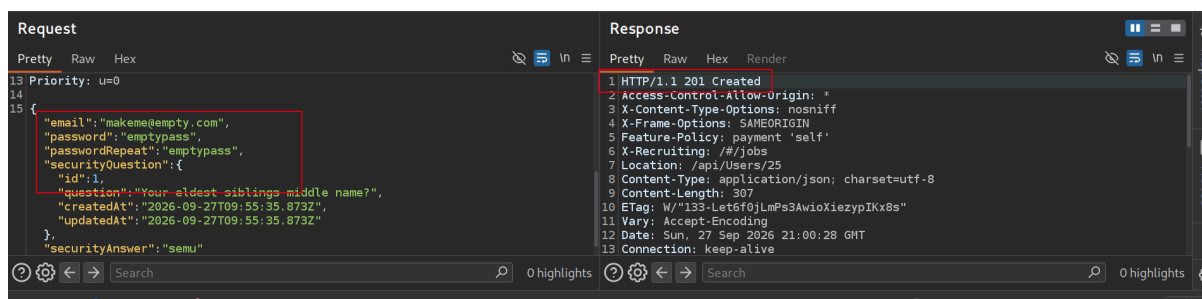


Figure 50. Evidence screenshot

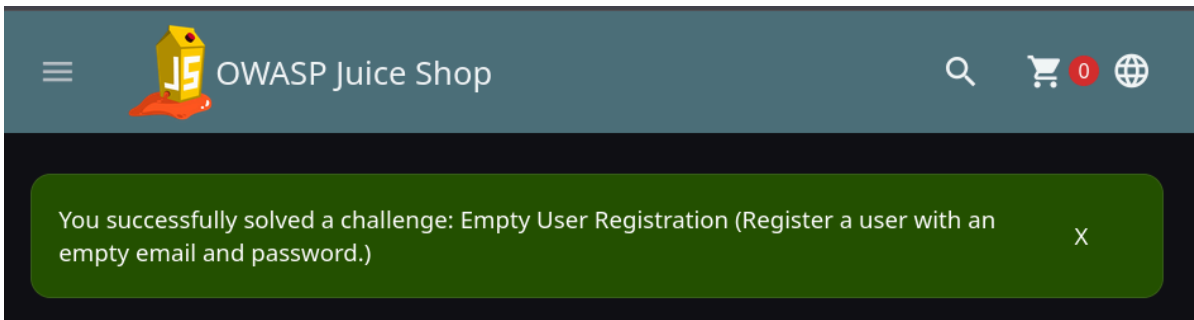


Figure 51. Evidence screenshot

Impact

- An exposed key may be abused to access a third-party service or impersonate the application.
- Depending on the provider and permissions, key abuse may result in unauthorized service usage, data exposure, or financial cost.
- Client-side exposure makes secrecy of the key impossible.

Recommended Remediation

1. Remove API keys and secrets from client-side code.
2. Store sensitive credentials on the server side using environment variables or a dedicated secrets manager.
3. Rotate any key that has been exposed.
4. Apply provider-side restrictions such as origin, API, scope, and quota controls where supported.

F-18: Improper Input Validation — Empty User Registration

Severity	Low/Medium
CVSS	Not specified
Affected Component	User Registration
Affected Endpoint	POST /api/Users/

Description

The registration endpoint allowed a user record to be created even when required authentication fields were modified to empty values. The server indicated that the values were invalid or required, but the record was nevertheless created.

2. Reject missing or empty email and password values.
3. Ensure failed validation prevents the database transaction from being committed.

F-19: Client-Side Exposure of User Credentials

Severity	Not specified in the source report
CVSS	Not specified
Affected Component	Client-side JavaScript
Affected Endpoint	http://localhost:3040/hacking-instructor-BXwB7EFQ.js

Description

Credentials were found hardcoded in a client-side JavaScript file. The source report indicates that searching the JavaScript for @juice-sh.op revealed credentials that could then be used to authenticate.

Reproduction / Verification Steps

1. Inspect the application product page using Developer Tools.
2. Open the Debugger/Sources view.
3. Search the JavaScript source for @juice-sh.op.
4. Identify the exposed credentials.
5. Attempt authentication with the discovered credentials and verify access.

Evidence

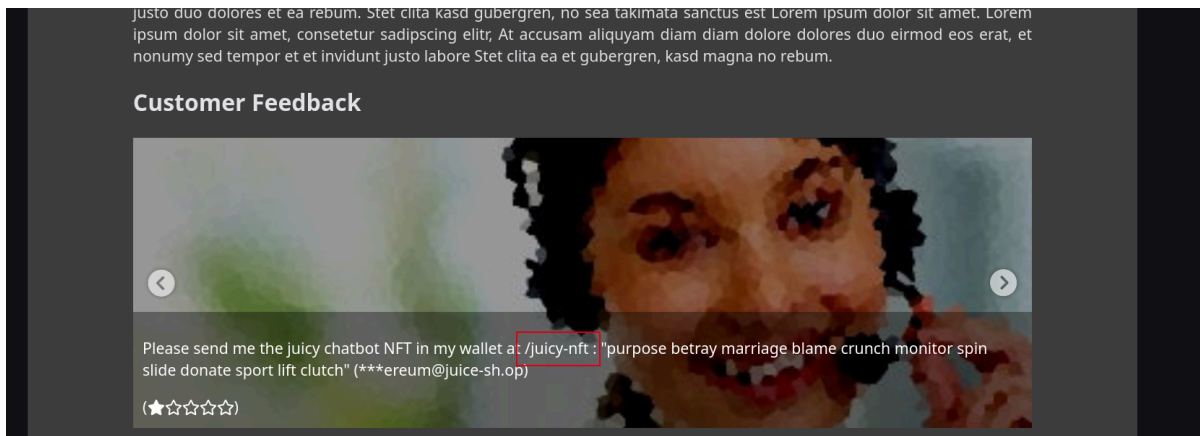


Figure 54. Evidence screenshot supplied with the original assessment record.

☒ Auto compute

Mnemonic Language [English](#) [日本語](#) [Español](#) [中文\(简体\)](#) [中文\(繁體\)](#) [Français](#) [Italiano](#) [한국어](#) [Čeština](#) [Português](#)

BIP39 Mnemonic

☐ Show split mnemonic cards

BIP39 Passphrase (optional)

BIP39 Seed

Figure 55. Evidence screenshot supplied with the original assessment record.

Impact

- Anyone able to load the client application can inspect and reuse the exposed credentials.
- The associated account can be accessed without the intended authentication secret.
- Compromised credentials may provide access to account data and functionality.

Recommended Remediation

1. Do not place credentials in JavaScript, HTML, comments, or other client-delivered content.
2. Move authentication secrets to protected server-side systems.
3. Immediately rotate credentials that have been exposed.

F-20: NFT Takeover Through Exposed Wallet Seed Phrase

Severity	Critical
CVSS	Not specified
Affected Component	About Page / Wallet Secret Handling
Affected Endpoint	/about

Description

A BIP-39 wallet seed phrase was exposed in an HTML comment on the /about page. Although the comment was not visible in the rendered page, it was accessible through the page source. The source report demonstrates that the seed could be used to derive the wallet private key and take control of the associated NFT.

Reproduction / Verification Steps

1. Open the /about page.
2. Inspect the page source and identify the wallet seed phrase in an HTML comment.

3. Use the exposed seed phrase with a BIP-39 derivation tool to generate the corresponding private key, as performed in the original assessment.
4. Use the derived private key in the /juice-nft challenge.
5. Verify that the NFT takeover challenge is completed.

Evidence

Table	CSV						
Path	Toggle	Address	Toggle	Public Key	Toggle	Private Key	Toggle
m/44' / 60' / 0' / 0/0		0x8343d2eb2813A2495De435a1b15e85b98115Ce05				0x5bcc3e9d38baa0e7bfabb80ae5957bbe8ef059e640311d7d6d465e6bc948e3e	
m/44' / 60' / 0' / 0/1		0x4A2d55CF9608085961974E6547C6d4F5E21b420E				0x7a63f1461d37b2591ac1381a44a6ababfe68fa2cdd5917c24a5e797103db22335	

Figure 56. Evidence screenshot supplied with the original assessment record.

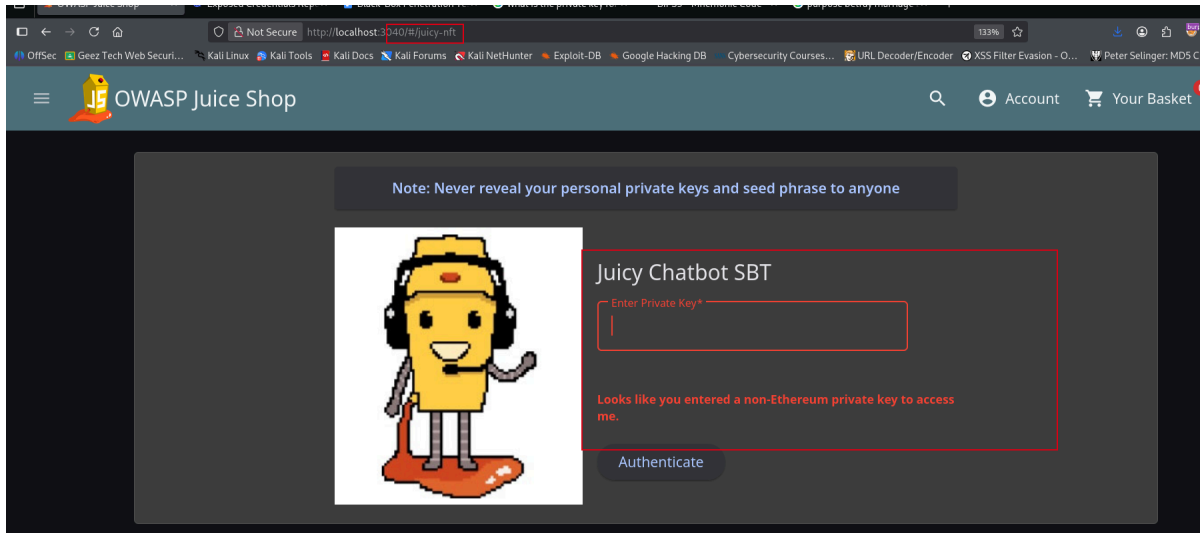


Figure 57. Evidence screenshot supplied with the original assessment record.

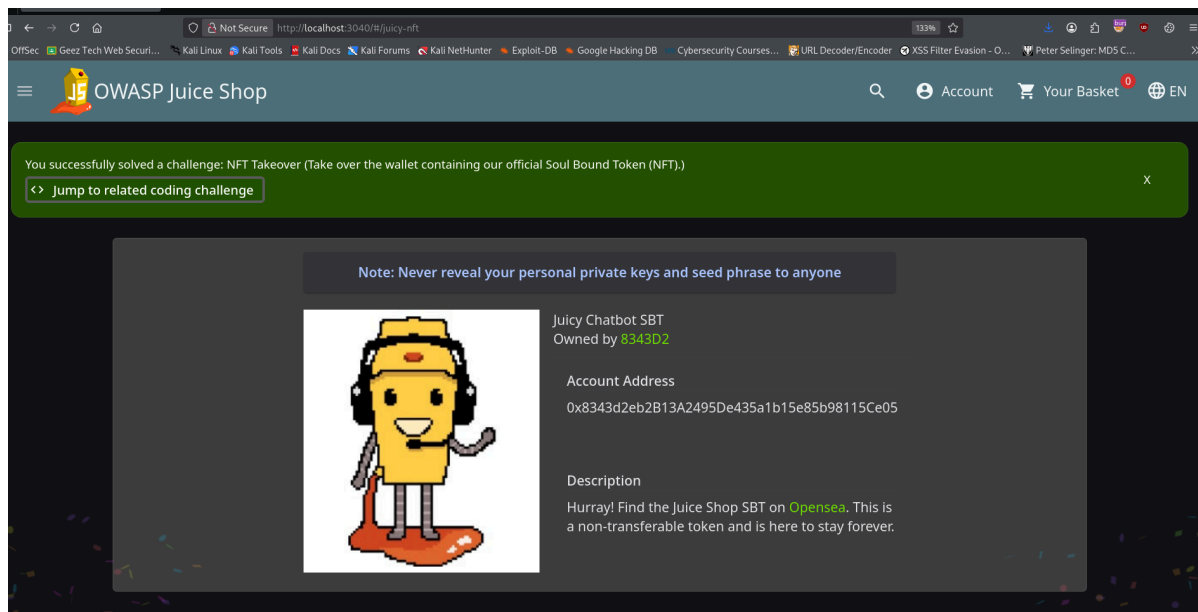


Figure 58. Evidence screenshot supplied with the original assessment record.

Impact

- An unauthenticated attacker can obtain the wallet secret and control the associated wallet/NFT.
- A compromised blockchain seed phrase cannot be revoked; the affected wallet must be replaced or its assets transferred to a new secure wallet.
- The exposure represents direct compromise of a high-value authentication secret.

Recommended Remediation

1. Remove the seed phrase from all application source, HTML, JavaScript, comments, and deployment artifacts.
2. Generate a new wallet and move affected assets to the replacement wallet.
3. Use a secrets-management process for operational keys.
4. Add secret-scanning controls to source control and CI/CD pipelines.

F-21: Sensitive Location Exposure Through Image EXIF Metadata

Severity	High
CVSS	Not specified
Affected Component	Photo Wall / Account Recovery
Affected Endpoint	/#/photo-wall and /#/forgot-password

Description

A publicly accessible image contained EXIF metadata with GPS information. The extracted location was used to answer the associated user's security question, demonstrating a path from public media metadata to account recovery.

Reproduction / Verification Steps

1. Open Forgot Password and submit `john@juice-sh.op`.
2. Observe the security question: "What's your favorite place to go hiking?"
3. Open the Photo Wall and download John's hiking photo.
4. Extract EXIF metadata from the image, for example with `exiftool`.
5. Use the GPS information to determine the corresponding location.
6. The source report identified the location as Daniel Boone National Forest.
7. Use the discovered information to answer the security question and test account recovery.

Evidence

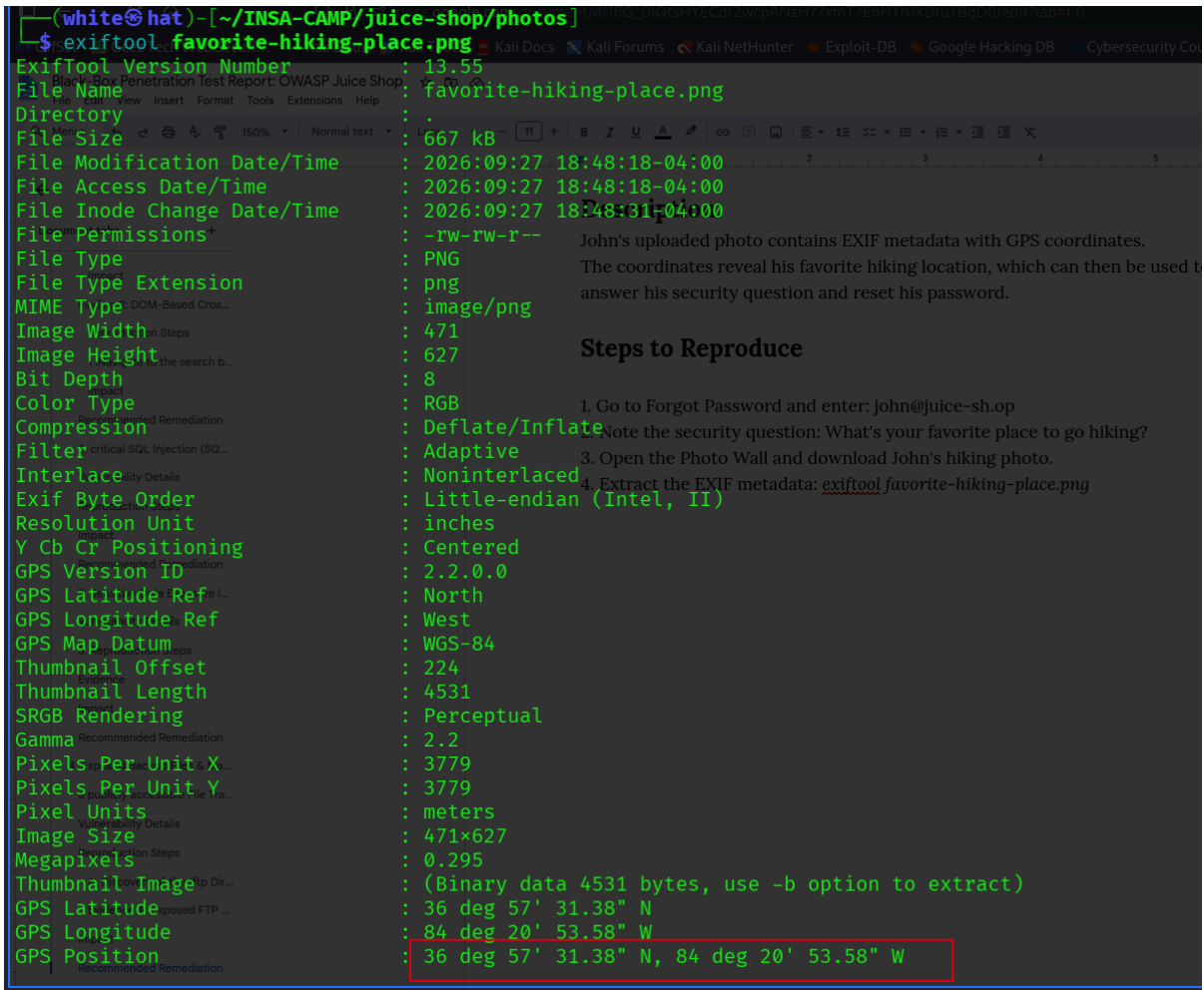


Figure 59. Evidence screenshot supplied with the original assessment record.

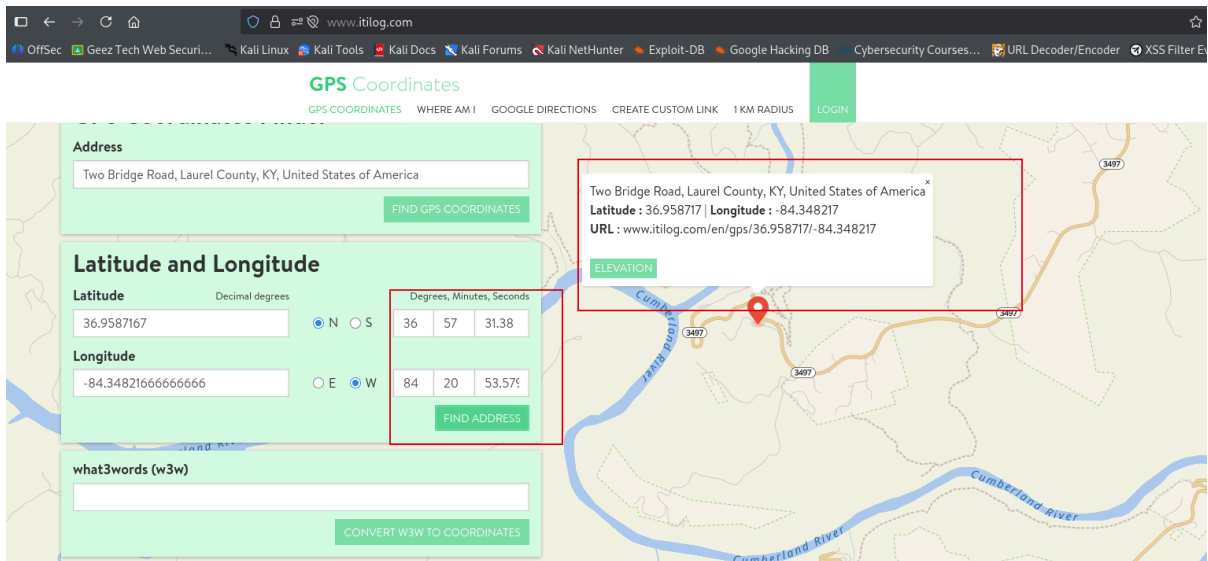


Figure 60. Evidence screenshot supplied with the original assessment record.

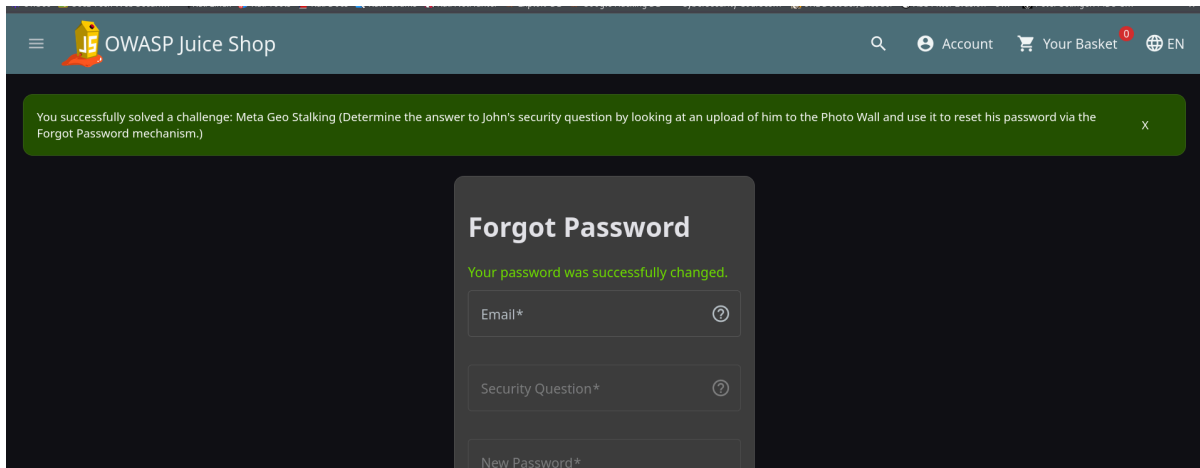


Figure 61. Evidence screenshot supplied with the original assessment record.

Impact

- Public image metadata can disclose sensitive location information.
- The disclosed location can be correlated with security questions.
- Successful exploitation may result in unauthorized password reset and account takeover.

Recommended Remediation

1. Strip EXIF and GPS metadata from user-uploaded images before public delivery.
2. Do not use knowledge-based security questions as the sole account-recovery factor.
3. Use verified email links, MFA, or another strong recovery mechanism.
4. Validate and sanitize uploaded files before storing and serving them.

F-22: Repetitive Registration — Insufficient Server-Side Password Confirmation

Severity	Medium
CVSS	Not specified
Affected Component	User Registration
Affected Endpoint	/#/register

Description

The Repeat Password field was validated primarily on the client side. After the initial values matched, changing the original password did not trigger effective revalidation, and the server accepted inconsistent password values.

Reproduction / Verification Steps

1. Open /#/register.
2. Enter test@test.com as the email.
3. Enter test123 in Password and Repeat Password.
4. Wait for the client-side password-match validation to pass.
5. Change the original Password value after validation has passed.
6. Submit the registration and observe whether the server revalidates the two password fields.

Evidence

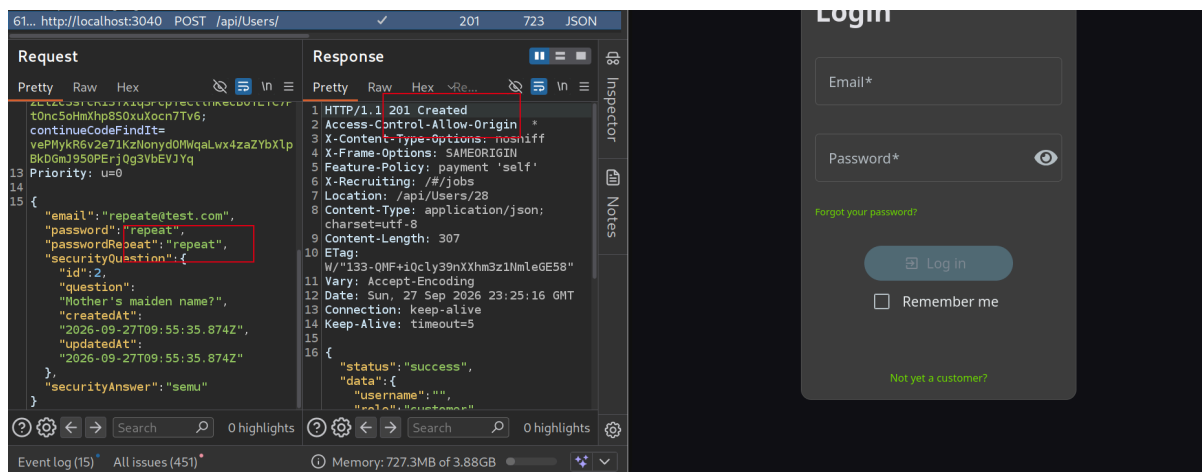


Figure 62. Evidence screenshot supplied with the original assessment record.

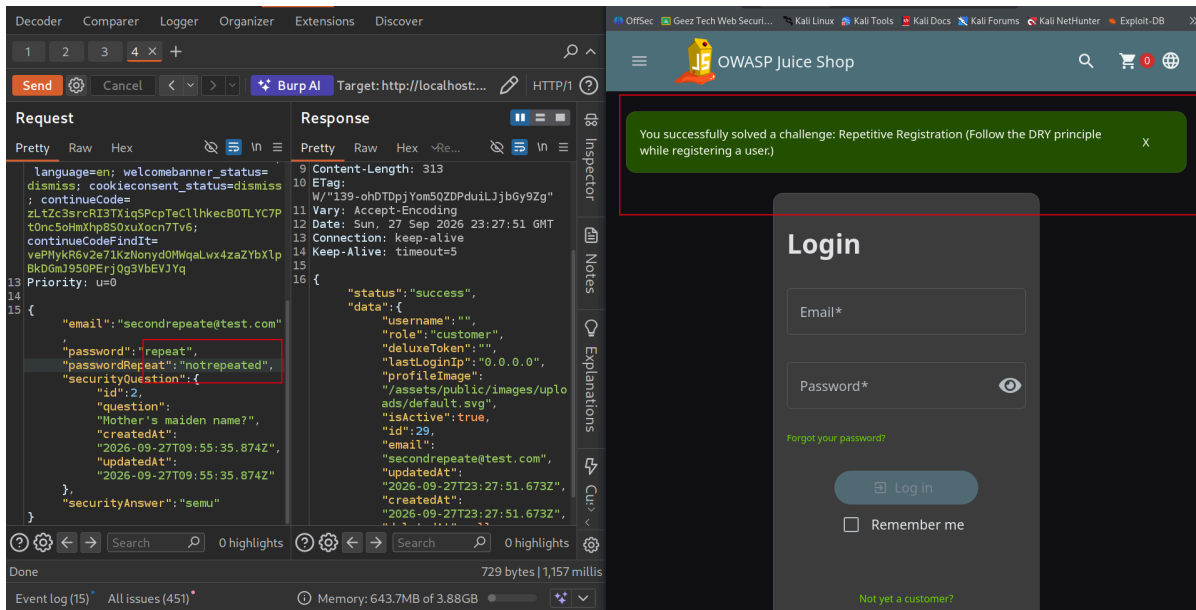


Figure 63. Evidence screenshot supplied with the original assessment record.

Impact

- The application may create accounts with inconsistent password-confirmation data.
- Client-side validation can be bypassed or become stale when dependent fields change.
- The finding demonstrates insufficient server-side validation of registration invariants.

Recommended Remediation

1. Validate password === passwordRepeat on the server for every registration request.
2. Treat client-side validation as a user-experience control only.
3. Revalidate dependent fields whenever their related values change.

F-23: Zero-Star Feedback — Improper Input Validation

Severity	Low
CVSS	Not specified
Affected Component	Customer Feedback
Affected Endpoint	POST /api/Feedbacks/ (UI: /#/contact)

Description

The feedback endpoint accepted a rating of zero after the client-side restriction was bypassed. The server did not enforce the expected rating range of 1–5.

Reproduction / Verification Steps

1. Open the Contact page at /#/contact.
2. Enter a comment and complete the CAPTCHA without selecting a star rating.
3. Use Developer Tools to locate the disabled Submit button.
4. Remove the disabled attribute from the button.
5. Submit the feedback.
6. Observe that the server accepts the feedback with a 0-star rating.

Evidence

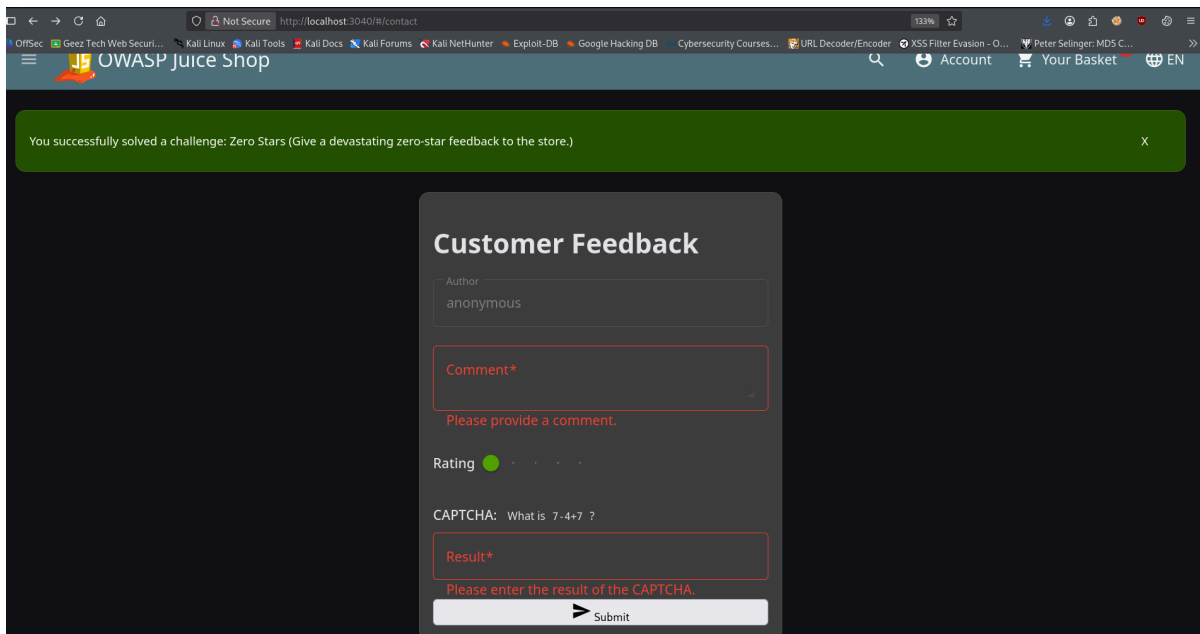


Figure 64. Evidence screenshot supplied with the original assessment record.

Impact

- Client-side validation can be bypassed to submit invalid data.
- Invalid ratings can affect data integrity and rating statistics.
- The issue demonstrates that server-side validation is not enforcing the business rule.

Recommended Remediation

1. Enforce the rating range of 1–5 on the server.
2. Reject invalid values with HTTP 400 Bad Request.
3. Do not rely on disabled HTML elements or other client-side controls as security boundaries.

F-24: Password Hash Exposure Through User Profile API

Severity	High
CVSS	Not specified
Affected Component	User Profile / API Response
Affected Endpoint	GET /rest/user/whoami

Description

The whoami endpoint allowed the client to request a password hash through the fields query parameter. By modifying an authenticated request, the assessment retrieved sensitive password data that should never be returned by the API.

Reproduction / Verification Steps

1. Log in to the application.
2. Intercept the request to /rest/user/whoami?fields=email using Burp Suite.
3. Send the request to Burp Repeater.
4. Modify the fields parameter to include the password field.
5. Observe that the API returns the password hash.

Evidence

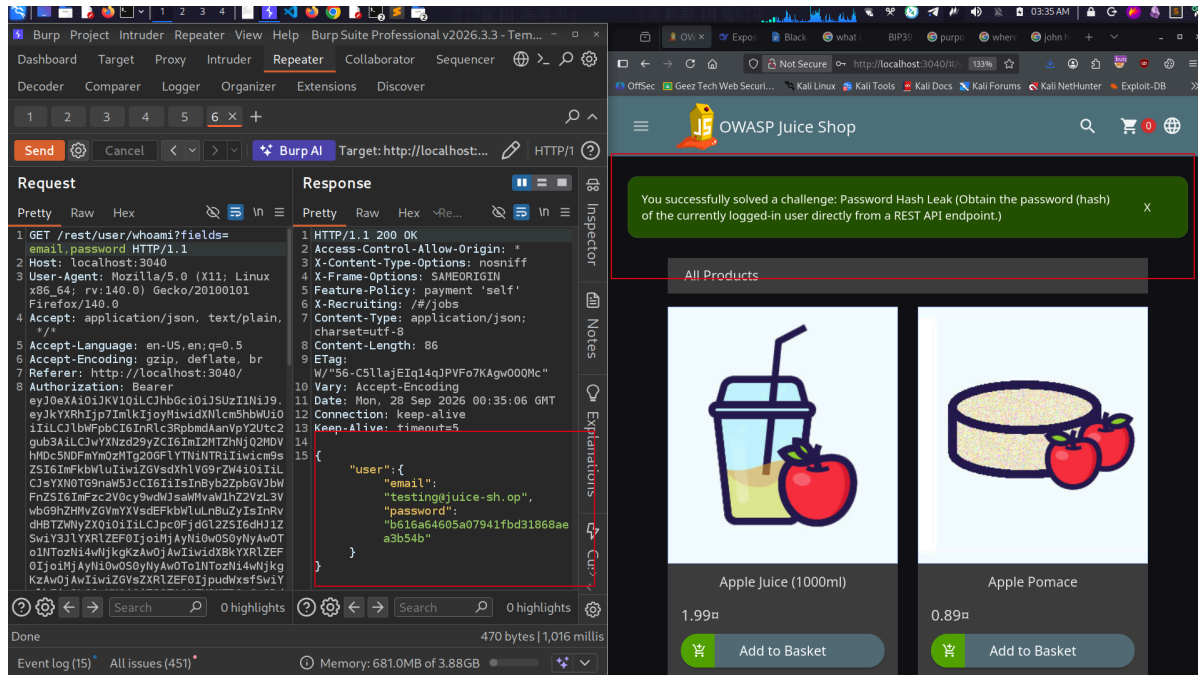


Figure 65. Evidence screenshot supplied with the original assessment record.

Impact

- An attacker with an authenticated session can retrieve a password hash.
- The hash can be subjected to offline password cracking.
- Password reuse across accounts or systems could increase the impact of a successful crack.

Recommended Remediation

1. Allow-list fields that may be returned by the endpoint.
2. Never expose password hashes through API responses.
3. Use Argon2id, bcrypt, or another dedicated password-hashing algorithm instead of MD5.
4. Review all API serializers to ensure sensitive authentication fields are excluded by default.

F-25: Visual Geo-Stalking — Sensitive Data Exposure

Severity	High
CVSS	Not specified
Affected Component	Photo Wall / Password Recovery
Affected Endpoint	/#/photo-wall and /#/forgot-password

Description

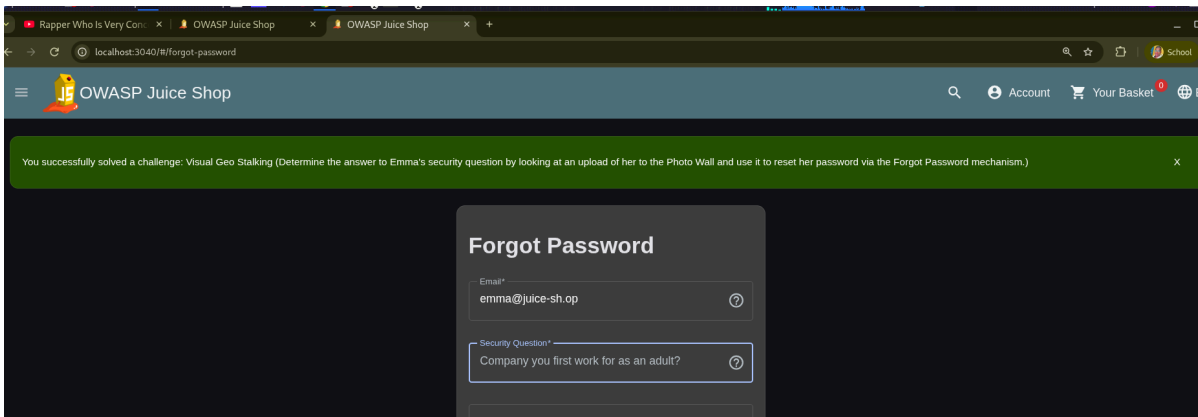
A publicly accessible photo exposed information about Emma's former workplace. The information was sufficient to answer the account's security question and complete a password-reset workflow.

Reproduction / Verification Steps

1. Open Forgot Password and enter emma@juice-sh.op.
2. Identify the security question: “Company you first work for as an adult?”
3. Open the Photo Wall and locate Emma's image captioned “My old workplace...”.
4. Inspect the image and identify the company name shown in the scene; the source report identified it as ITsec.
5. Return to the password-recovery process and enter ITsec as the security-question answer.
6. Set a new password and submit the recovery request.

Evidence



Figure 66. Evidence screenshot supplied with the original assessment record.*Figure 67. Evidence screenshot supplied with the original assessment record.*

Impact

- Publicly available personal information can be used to answer account-recovery questions.
- Successful exploitation can result in unauthorized password reset and account takeover.
- The finding demonstrates the weakness of knowledge-based recovery questions.

Recommended Remediation

1. Replace knowledge-based security questions with secure recovery links sent to verified channels.
2. Require additional verification for sensitive account-recovery actions.
3. Educate users about the privacy risks of publishing identifiable personal information.

F-26: Weak Password Cryptography — Use of Unsalted MD5

Severity	High
CVSS	Not specified
Affected Component	Password Storage / User Data
Affected Endpoint	/rest/user/whoami?fields=email,password

Description

The application stores user passwords using MD5 without unique salts. MD5 is a fast, obsolete hashing algorithm and is not suitable for password storage because it enables efficient offline guessing.

Reproduction / Verification Steps

1. Log in to the application.
2. Inspect authenticated user data using Burp Suite.
3. Request `/rest/user/whoami?fields=email,password`.
4. Observe that the password is represented as a 32-character hexadecimal MD5 hash.
5. The source report recorded an example hash of `0192023a7bbd73250516f069df18b500` and identified the algorithm as MD5.

Evidence

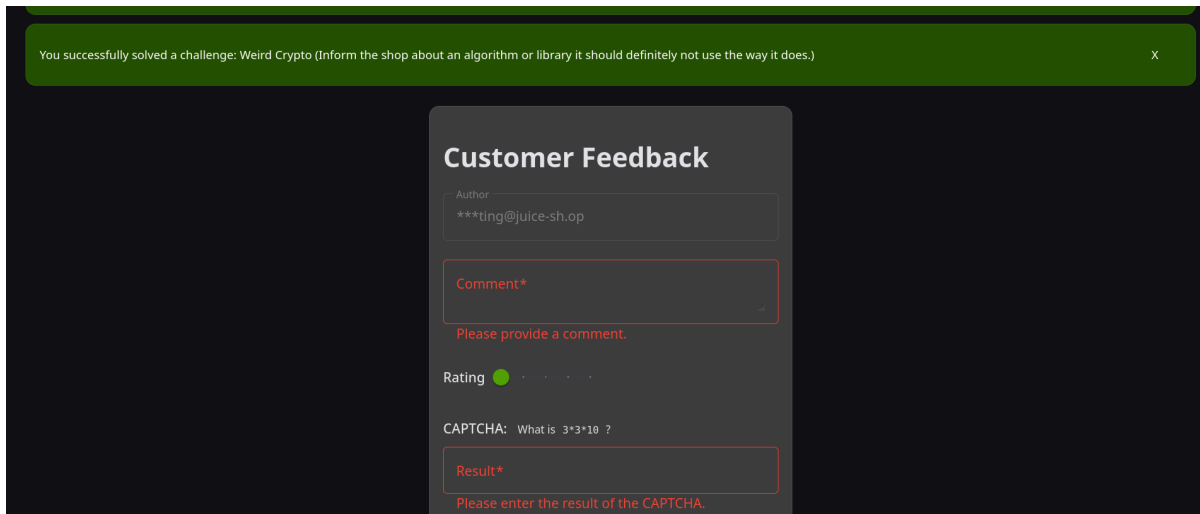


Figure 68. Evidence screenshot supplied with the original assessment record.

Impact

- Exposed hashes can be attacked using offline dictionary and brute-force techniques.
- The absence of unique salts allows identical passwords to produce identical hashes.
- Successful cracking can result in account compromise and credential reuse attacks.

Recommended Remediation

1. Replace MD5 with Argon2id, bcrypt, or another dedicated password-hashing algorithm.
2. Generate a unique cryptographically random salt for every password.

3. Migrate existing password hashes to the secure scheme using a controlled password-reset or rehash process.
4. Never return password hashes through application APIs.

F-27: Login Authentication Bypass for Jim via SQL Injection

Severity	High
CVSS	Not specified
Affected Component	Login / Authentication
Affected Endpoint	/#/login

Description

The login endpoint was vulnerable to SQL injection, allowing the tester to manipulate the email parameter and bypass the password verification for the Jim account.

Reproduction / Verification Steps

1. Identify the Jim account email from client-side application data: jim@juice-sh.op.
2. Open the login page.
3. Enter the SQL injection value jim@juice-sh.op'-- in the email field.
4. Enter any value in the password field.
5. Submit the login request and observe that the password portion of the SQL statement is effectively bypassed.

Evidence

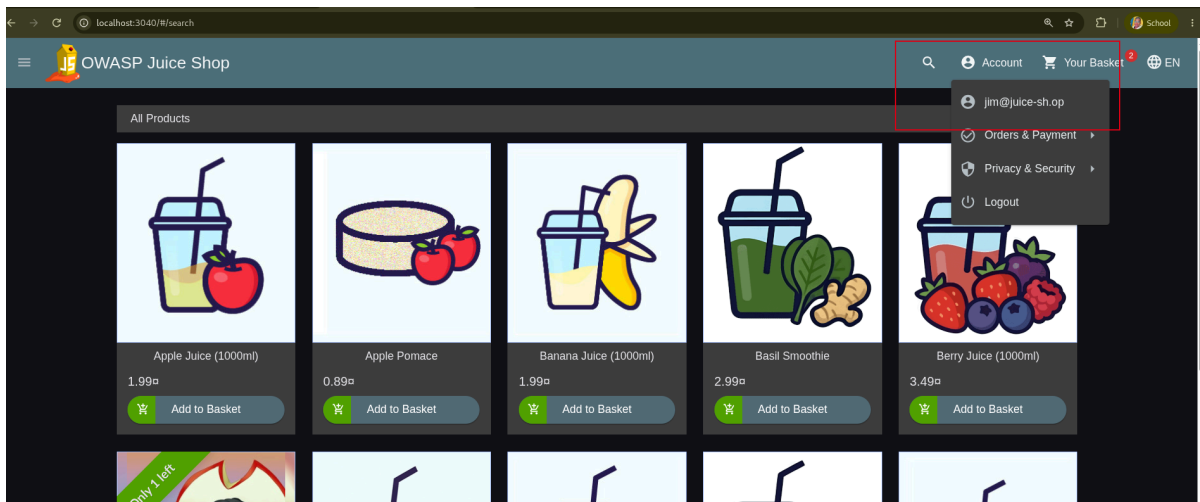


Figure 69. Evidence screenshot supplied with the original assessment record.

Impact

- An attacker can authenticate as another user without knowing that user's password.
- Successful exploitation provides unauthorized access to the targeted account and its associated data.

Recommended Remediation

1. Use parameterized queries or prepared statements.
2. Never concatenate user-controlled input directly into SQL statements.
3. Apply server-side input validation as a secondary control.
4. Use ORM parameterization features where appropriate.

F-28: Login Authentication Bypass for Bender via SQL Injection

Severity	High
CVSS	Not specified
Affected Component	Login / Authentication
Affected Endpoint	/#/login

Description

The same SQL injection condition affecting authentication was demonstrated against the Bender account. A crafted email value containing a SQL comment sequence caused the password portion of the authentication query to be disregarded.

Reproduction / Verification Steps

1. Identify the Bender account email from client-side application data.
2. Open the login page.
3. Enter `bender@juice-sh.op'--` in the email field.
4. Enter any value in the password field.
5. Submit the request and observe that authentication succeeds without the valid password.

Evidence

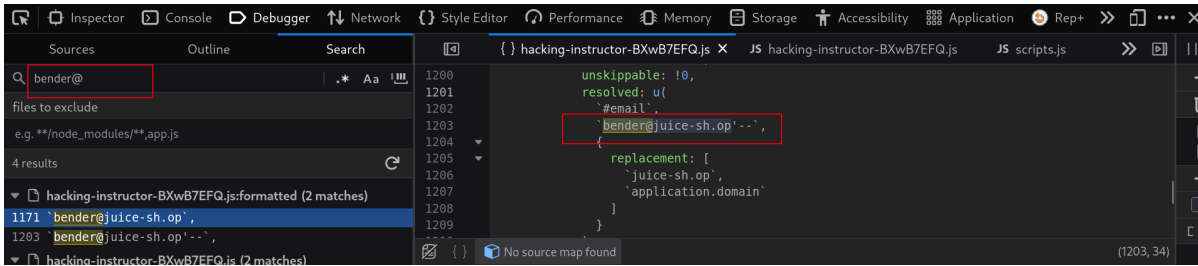


Figure 70. Evidence screenshot supplied with the original assessment record.

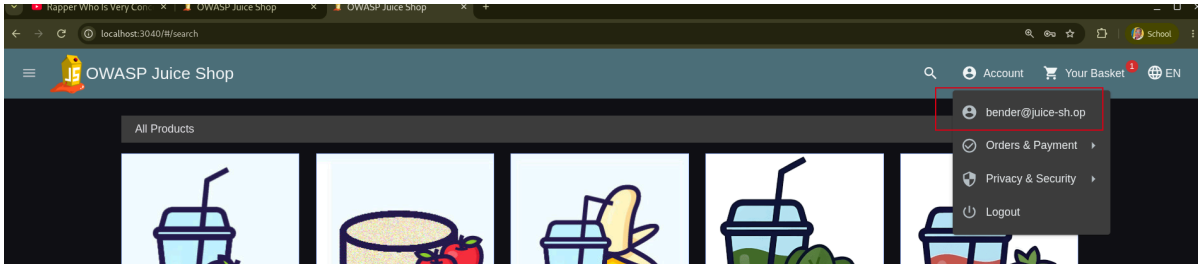


Figure 71. Evidence screenshot supplied with the original assessment record.

Impact

- An attacker can bypass authentication for another user's account.
- Successful exploitation exposes the targeted account and demonstrates that the authentication boundary is not reliably enforced.

Recommended Remediation

1. Use parameterized queries or prepared statements.
2. Never concatenate user-controlled values into SQL queries.
3. Validate inputs as a secondary defense.
4. Use secure ORM query mechanisms and regression tests for authentication bypass.

F-29: CAPTCHA Bypass

Severity	Medium
CVSS	Not specified
Affected Component	CAPTCHA / Feedback Submission
Affected Endpoint	GET /rest/captcha/ and feedback submission

Description

The CAPTCHA endpoint exposed the expected answer in its API response and accepted previously used CAPTCHA values. This allows an attacker to automate or bypass the CAPTCHA control.

Reproduction / Verification Steps

1. Intercept the CAPTCHA response with Burp Suite.
2. Inspect the response and observe that the correct CAPTCHA answer is included.
3. Submit a feedback request and send it to Burp Intruder.
4. Reuse the exposed or previously accepted CAPTCHA value and observe that the submission is accepted.

Evidence

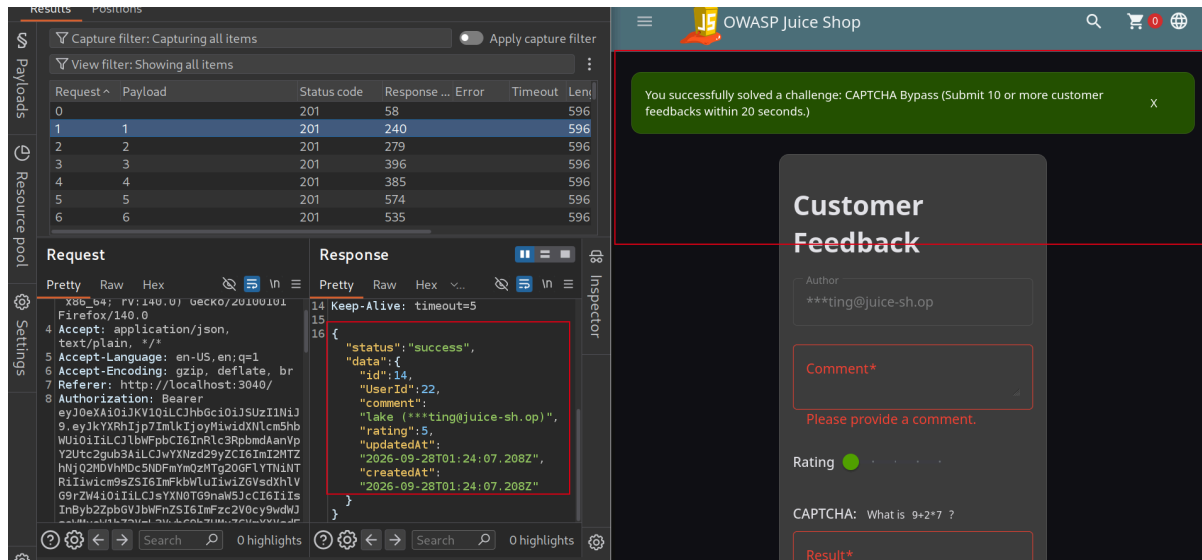


Figure 72. Evidence screenshot supplied with the original assessment record.

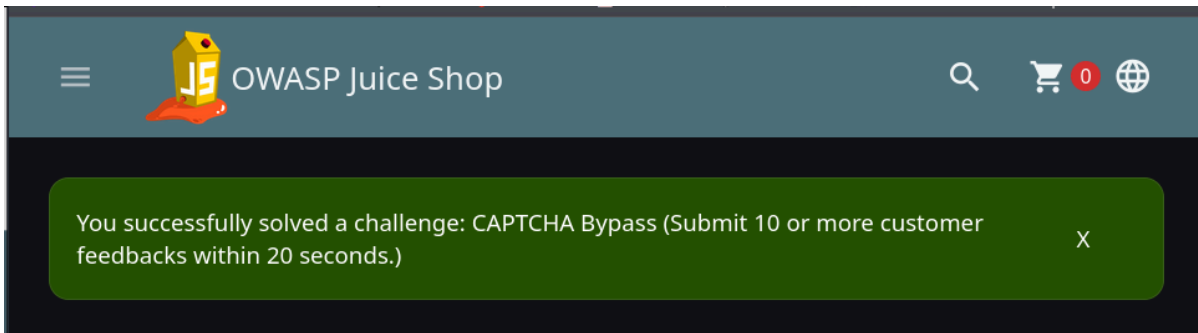


Figure 73. Evidence screenshot supplied with the original assessment record.

Impact

- Automated feedback submissions become easier.
- Attackers may generate spam and consume application resources.
- The CAPTCHA no longer provides a reliable server-side anti-automation control.

Recommended Remediation

1. Never return the CAPTCHA answer to the client.
2. Use server-side CAPTCHA verification tied to a short-lived, single-use challenge token.
3. Invalidate each CAPTCHA token immediately after successful or failed verification.
4. Apply server-side rate limiting to the protected operation.

F-30: Missing URL Encoding Allows Access to an Unintended Resource

Severity	Not specified in the source report
CVSS	Not specified
Affected Component	Photo Wall / Image Resource Handling
Affected Endpoint	Image URL/file parameter

Description

A resource path contained literal hash characters that were not correctly URL-encoded. Because the browser interprets # as a fragment delimiter, the intended resource could not be loaded normally. Encoding the characters as %23 allowed the tester to retrieve the intended image.

Reproduction / Verification Steps

1. Open the Photo Wall and observe the missing image in the affected position.
2. Inspect the HTML and identify the image source containing literal # characters.
3. Observe the source pattern containing values such as #zatschi and #whoneedsfourlegs.
4. URL-encode the hash characters as %23.
5. Reload the resource using the encoded path and verify that the image is retrieved.

Evidence

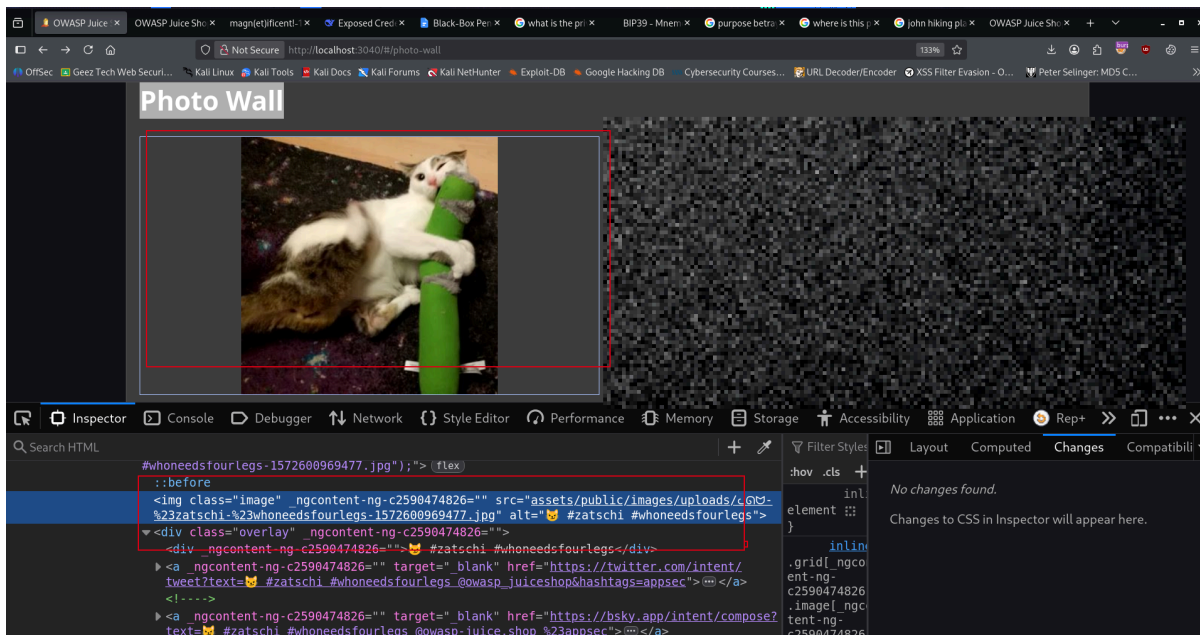


Figure 74. Evidence screenshot supplied with the original assessment record.

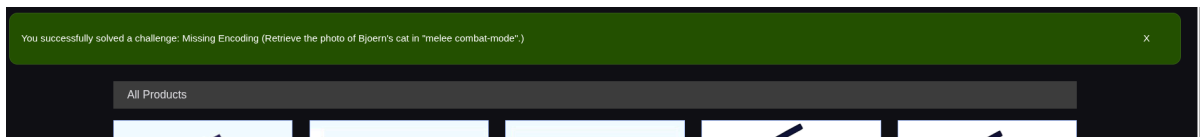


Figure 75. Evidence screenshot supplied with the original assessment record.

Impact

- Improper encoding can alter the interpretation of URL and file parameters.
- Parameter parsing weaknesses can lead to unintended resource access or broken security controls.
- The issue demonstrates insufficient canonicalization and encoding of resource identifiers.

Recommended Remediation

1. Apply correct URL encoding whenever user-controlled or special-character data is placed into URLs.
2. Canonicalize and validate file/resource identifiers server-side.
3. Do not rely on browser parsing behavior as a security control.

F-31: Business Logic Bypass — Deluxe Membership Payment

Validation

Severity	High
CVSS	Not specified
Affected Component	Deluxe Membership / Payment Processing
Affected Endpoint	POST /rest/deluxe-membership

Description

The Deluxe Membership workflow granted membership without verifying a successful payment. During testing, the disabled wallet payment control was enabled in the browser and the payment request was intercepted and modified. The server accepted an empty paymentMode and granted membership.

Reproduction / Verification Steps

1. Log in as a standard user and navigate to the Deluxe Membership page.
2. Use browser Developer Tools to enable the disabled wallet payment control.
3. Intercept the membership/payment request with Burp Suite.
4. Modify the request body so that the paymentMode is empty, as demonstrated during the assessment.
5. Submit the modified request and observe whether membership is granted without a verified transaction.

Evidence

Impact

- Users can potentially obtain paid membership benefits without payment.
- The issue can cause direct financial loss or unauthorized access to premium functionality.
- The finding demonstrates that the server relies on client-side payment state instead of independently validating the transaction.

1. Perform all payment and entitlement validation server-side.
2. Reject empty or unsupported payment methods.
3. Grant membership only after receiving and verifying a successful payment transaction.
4. Do not use disabled UI controls as an authorization or payment-security boundary.

F-32: Publicly Accessible Support Logs and Observability Data

Severity	Not specified in the source report
CVSS	Not specified
Affected Component	Support Logs / Web Server Configuration
Affected Endpoint	GET /support/logs

Description

Server access and audit logs were stored in a web-accessible directory with directory listing enabled. The listing exposed log files containing application activity information such as request paths, IP addresses, and user-agent data.

Reproduction / Verification Steps

1. Open a web browser.
2. Navigate to `http://localhost:3040/support/logs`.
3. Observe the directory listing.
4. The source report identified files including `access.log.2026-09-27`, `access.log.2026-09-28`, and `audit.json`.
5. Download or inspect the exposed files and review the information they contain.

Evidence

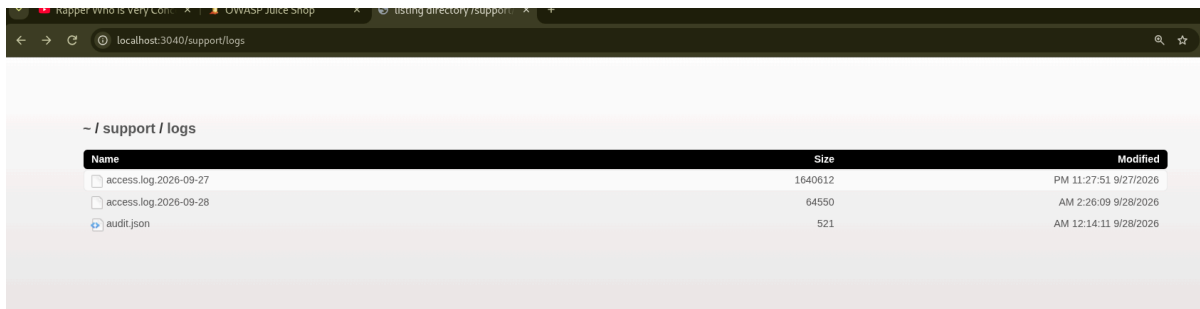


Figure 77. Evidence screenshot supplied with the original assessment record.

Impact

- Attackers can use log information to map valid endpoints and application behavior.

- Logs may reveal IP addresses, request paths, user agents, internal addresses, session tokens, API keys, or other sensitive data if such information was logged.
- Exposed operational data can significantly improve an attacker's reconnaissance capability.

Recommended Remediation

1. Restrict access to the /support directory and any directory containing operational logs.
2. Disable directory listing.
3. Store logs outside the web root and access them through a protected logging system.
4. Sanitize logs so that passwords, tokens, API keys, and other secrets are never recorded.
5. Apply least-privilege access controls to operational logging infrastructure.

F-33: Unsigned JWT Accepted by Authentication Logic

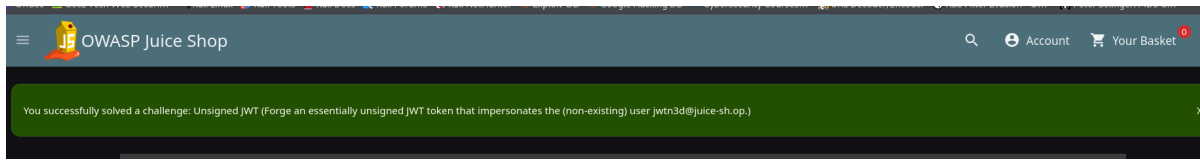
Severity	Not specified in the source report
CVSS	Not specified
Affected Component	JWT Validation Logic
Affected Endpoint	POST /rest/user/login and protected API requests

Description

The application accepted a JWT whose algorithm was set to none and did not require a valid cryptographic signature. The assessment modified a valid token and changed the identity information, demonstrating that an attacker could potentially impersonate another user without knowing a password or signing secret.

Reproduction / Verification Steps

1. Authenticate as a normal user and capture a valid JWT from the login response or client storage.
2. Decode the JWT header and payload.
3. Modify the payload identity, including the email field, to the target identity.
4. Set the JWT header algorithm to none.
5. Supply the modified token to the application through the token value used for authentication.
6. Observe that the application accepts the modified token and processes the request as the claimed identity.

Figure 80. Evidence screenshot supplied with the original assessment record.*Figure 81. Evidence screenshot supplied with the original assessment record.*

Impact

- Authentication can be bypassed by forging a token without a valid signature.
- An attacker may impersonate ordinary users or administrators.
- Successful exploitation can result in account takeover, data theft, and unauthorized privileged functionality.
- The issue undermines the application's entire JWT-based authentication boundary.

Recommended Remediation

1. Explicitly reject tokens using the none algorithm.
2. Configure the JWT library with an allow-list of approved algorithms, such as RS256 where appropriate.
3. Verify the cryptographic signature before trusting any JWT claim.
4. Never allow the client to select the verification algorithm.
5. Use a strong, secret, and non-guessable signing key and protect it through secure secret management.
6. Add automated tests covering algorithm-confusion and unsigned-token attacks.

6. Conclusion

The assessment evidence demonstrates that the application has weaknesses across several security control layers. In particular, the documented SQL injection and JWT issues undermine authentication and authorization, while exposed credentials, password hashes, wallet secrets, public logs, and backup files increase the amount of sensitive information available to an attacker.

Remediation should be prioritized around server-side security boundaries: parameterized database access, strict authentication and JWT validation, server-side authorization and ownership checks, secure password hashing, removal and rotation of exposed secrets, protected file and log storage, and robust server-side input validation. Client-side controls should be treated as usability features rather than security boundaries.